



CONSERVATOIRE NATIONAL DES ARTS ET MÉTIERS

Master Finance de marché

**L'approche agentique dans un contexte de trading
multifactoriel**

Synthèse LLM et allocation de portefeuille sous contraintes

Nom et prénom : SANOU Abou Dramane

Encadrant : Bertrand DJEMBISSI

Rapport d'étude présenté dans le cadre du Master Finance de marché

Paris, septembre 2026

Résumé

Ce mémoire étudie deux usages d'un LLM en trading multifactoriel : restituer des résultats et proposer une allocation simulée sous contraintes. Le prototype initial associe un backtest Python, LangGraph et une synthèse DeepSeek. Dans l'extension, Python valide les poids proposés et calcule le portefeuille. Aucun outil d'ordre réel n'est accessible.

Quatre ETF représentent les expositions value, momentum, quality et low volatility : VLUE, MTUM, QUAL et USMV. Sur février 2015–décembre 2025, leur allocation équi-pondérée à 10 points de base atteint 12,02 % de rendement annualisé, contre 13,84 % pour SPY. Le contrôle temporel sur 2021–2025 conserve ce classement.

L'évaluation de la synthèse comprend 12 cas, dont 11 répétés trois fois et un cas qualité bloqué. Les 33 réponses passent les contrôles automatisés. Toutefois, trois altérations construites sont également acceptées : un coût nul, une citation inexistante et une recommandation contradictoire. La conformité mesurée ne garantit donc pas la fiabilité générale des notes.

L'extension d'allocation couvre mai 2021–décembre 2025 : 22 des 56 propositions sont exécutées et 34 rejetées, surtout pour dépassement du turnover. Pour 100 unités initiales, DeepSeek termine à 148,39, contre 153,55 pour l'équipondération et 152,26 pour l'inverse volatilité, après frais de transaction. Le replay reproduit ces résultats. Cette expérience rétrospective, exposée au biais de préentraînement et hors coûts API et humains, ne démontre pas d'avantage financier de l'agent.

Mots-clés : agents intelligents, trading multifactoriel, LLM, explicabilité, gouvernance, back-testing, LangGraph.

Abstract

This report examines two uses of an LLM in multifactor trading : summarizing results and proposing constrained allocations in simulation. The initial prototype combines a Python backtest, LangGraph and a DeepSeek summary. In the extension, Python validates proposed weights and computes portfolio values. No live order tool is available.

Four ETFs represent value, momentum, quality and low volatility : VLU, MTUM, QUAL and USMV. Over February 2015–December 2025, their equal-weighted portfolio with 10-basis-point transaction costs achieves a 12.02% annualized return, versus 13.84% for SPY. A temporal check over 2021–2025 preserves this ranking.

The summary evaluation covers 12 cases, including 11 repeated three times and one blocked data-quality case. All 33 responses pass automated checks. However, three constructed alterations also pass : a zero cost, a nonexistent citation and a contradictory recommendation. Measured compliance therefore does not guarantee general reliability.

The allocation extension covers May 2021–December 2025 : 22 of 56 proposals are executed and 34 rejected, mainly for excessive turnover. Starting from 100 units, DeepSeek ends at 148.39, versus 153.55 for equal weighting and 152.26 for inverse volatility, after transaction costs. Archived replay reproduces these results. This retrospective experiment is exposed to pretraining look-ahead bias and excludes API and human costs ; it does not establish a financial advantage for the agent.

Keywords : intelligent agents, multifactor trading, LLM, explainability, governance, backtesting, LangGraph.

Remerciements

Je remercie mon encadrant, Bertrand DJEMBISSI, pour son accompagnement et ses orientations dans la préparation de ce rapport d'étude. Je remercie également l'équipe pédagogique du CNAM pour le cadre de formation fourni au sein du Master Finance de marché.

Déclaration relative à l'utilisation de l'intelligence artificielle

Dans le cadre de la préparation de ce rapport, j'ai utilisé des outils d'intelligence artificielle générative comme assistants de rédaction, de programmation et de vérification. Ils m'ont notamment aidé à corriger des fautes d'orthographe et de grammaire, à améliorer certaines formulations, à repérer des incohérences de notation et à vérifier la cohérence apparente de formules, de pseudo-codes et de portions de code. Cette assistance a également été mobilisée pour développer l'extension d'allocation, préparer ses tests, exécuter les simulations et rédiger leur analyse. Les contrôles automatisés et le replay sont documentés à partir du code, des équations et des archives ; ils ne valent pas validation indépendante de toutes les hypothèses financières. L'intelligence artificielle n'a donc pas été considérée comme une source autonome de résultats : les choix méthodologiques, l'analyse, l'interprétation et la validation finale présentés dans ce mémoire relèvent de ma responsabilité.

Table des matières

	Page
Résumé	i
Abstract	ii
Remerciements	iii
Déclaration relative à l'utilisation de l'intelligence artificielle	iv
Liste des figures	ix
Liste des tableaux	x
Glossaire	xi
1 Introduction et problématique	1
1.1 Contexte général	1
1.2 Problématique	2
1.3 Objectifs du rapport	2
1.4 Périmètre	3
1.5 Organisation du rapport	3
2 État de l'art et solutions GitHub	4
2.1 Définir l'approche agentique	4
2.2 État des lieux synthétique du domaine	5
2.3 Trading multifactoriel : chaîne quantitative de référence	5

2.4	Frameworks généralistes d’agents	6
2.5	Plateformes quantitatives et briques de trading ouvertes	6
2.6	Fiches techniques des briques étudiées	7
2.6.1	LangGraph : orchestration et état	7
2.6.2	Qlib : recherche quantitative et backtesting	8
2.6.3	OpenBB : accès aux données et veille	8
2.7	Projets GitHub spécifiquement agentiques en finance	8
2.8	Critères observables et statut dans le mémoire	9
2.9	Lecture critique de l’état de l’art	10
2.10	Hierarchie des sources	10
3	Méthodologie d’analyse	12
3.1	Démarche documentaire	12
3.2	Revue réglementaire arrêtée au 30 août 2026	13
3.3	Grille d’évaluation et scoring	14
3.4	Principes de conception	16
3.5	Méthode de classement des solutions	16
3.6	Protocole expérimental initial	17
3.7	Protocole d’évaluation de la couche LLM	19
3.7.1	Analyse complémentaire des notes et du validateur	21
3.8	Extension : décisions d’allocation et PnL simulé	21
3.8.1	Période, références et informations disponibles	22
3.8.2	Contrat, contraintes et traitement d’un rejet	22
3.8.3	Exécution différée et frais autofinancés	23
3.8.4	Configuration du modèle et budget de collecte	24
3.8.5	Journalisation et reproduction	25
3.9	Limites de la méthode	25
4	Architecture cible pour un desk multifactoriel agentique	26
4.1	Principe directeur	26
4.2	Vue d’ensemble de l’architecture	26

4.3	Socle quantitatif déterministe	27
4.4	Agents proposés	28
4.5	Flux de décision et boucle de rejet	29
4.6	Journal de décision explicable	30
4.7	Monitoring continu	31
4.8	Exemple de scénario de rééquilibrage	32
4.9	Composant d'allocation effectivement expérimenté	33
5	Résultats expérimentaux et discussion	34
5.1	Résultats du prototype expérimental initial	34
5.2	Démonstration de la couche agentique	36
5.3	Évaluation automatisée de la couche LLM	37
5.4	Comparaison d'une note déterministe et d'une synthèse LLM	39
5.4.1	Un même état quantitatif, deux restitutions	39
5.4.2	Grille de lecture qualitative et variabilité	40
5.4.3	Conditions d'utilité pour un analyste	42
5.5	Contre-exemples construits pour éprouver le validateur	42
5.6	Résultats de l'allocation proposée par DeepSeek	44
5.6.1	Variations selon les années	45
5.6.2	Propositions reçues, exécutées et rejetées	46
5.6.3	Coûts de transaction et ressources du service LLM	47
5.6.4	Vérifications obtenues	48
5.6.5	Biais et portée de l'interprétation	49
5.7	Apports de l'approche agentique	50
5.8	Risques spécifiques	51
5.9	Explicabilité : trois niveaux	52
5.9.1	Explicabilité quantitative	52
5.9.2	Explicabilité agentique	52
5.9.3	Explicabilité de la décision finale	52
5.10	Positionnement des solutions GitHub	53

5.11	Conditions minimales de mise en production	53
6	Conclusion	54
6.1	Ce que je retiens du prototype	54
6.2	Limites de portée	55
6.3	Deux suites prioritaires	55
	Bibliographie	56
	Annexes	60
I	Matrice des solutions GitHub	61
I.1	Solutions recommandées pour l'étude comparative	61
I.2	Due diligence des dépôts au 30 août 2026	62
I.3	Statut d'utilisation dans le mémoire	63
I.4	Combinaison minimale	64
II	Checklist de gouvernance d'un agent de trading	65
II.1	Avant expérimentation	65
II.2	Pendant le backtesting	65
II.3	Avant passage en observation	66
II.4	Avant toute exécution réelle	66
III	Reproduction de l'expérience d'allocation	67
III.1	Fichiers et vérifications	67
III.2	Commandes de reproduction sans appel API	68

Liste des figures

	Page
3.1 Déroulement du protocole expérimental et contrôle hors échantillon.	19
4.1 Architecture cible séparant le socle quantitatif, les agents et la gouvernance. .	27
4.2 Permissions, escalades et interdictions dans la couche agentique.	29
4.3 Séquence de décision avec boucle de rejet par le contrôle risque.	30
4.4 Extrait réel du journal JSON produit par le prototype.	31
4.5 Monitoring agentique et réactions en cas de dépassement de seuil.	32
5.1 Évolution d'un investissement initial de 100 unités.	35
5.2 Drawdown du benchmark et des portefeuilles multifactoriels.	36
5.3 Capital et drawdown des trois politiques d'allocation, de mai 2021 à décembre 2025. Frais de 10 pb par unité de turnover ; valeurs quotidiennes.	45
5.4 Trois niveaux d'explicabilité reliés au journal d'audit.	52

Liste des tableaux

	Page
2.1 Frameworks agentiques généralistes disponibles sur GitHub.	6
2.2 Briques quantitatives GitHub utiles pour une étude multifactorielle.	7
2.3 Solutions GitHub orientées agents financiers.	9
2.4 Critères observables et statut des principales solutions.	10
3.1 Grille de scoring des solutions agentiques et quantitatives.	15
3.2 Scoring documentaire provisoire des dépôts retenus.	15
4.1 Rôles, permissions et limites des agents proposés.	28
4.2 Séquence de décision et conditions de rejet.	29
4.3 Métriques agentiques, seuils d’alerte et réactions.	32
5.1 Résultats du prototype multifactoriel et du benchmark.	34
5.2 Contrôle hors échantillon sur la période 2021-2025.	35
5.3 Évaluation des deux démonstrations agentiques.	37
5.4 Résultats selon les contrôles automatisés du protocole LLM.	38
5.5 Extraits des deux restitutions du 31 décembre 2025.	40
5.6 Grille de lecture des restitutions sur une même date.	41
5.7 Résultat du validateur sur trois altérations et deux témoins.	43
5.8 Performance de l’expérience d’allocation, mai 2021–décembre 2025.	44
5.9 Rendements nets par année de l’expérience d’allocation.	45
5.10 Motifs de rejet des propositions DeepSeek.	46
5.11 Turnover exécuté et frais déduits des portefeuilles, capital initial 100.	47
5.12 Ressources rapportées par l’API pendant la collecte d’allocation.	47
5.13 Risques agentiques, contrôles et responsabilités.	51
I.1 Matrice de sélection des dépôts GitHub.	62
I.2 Grille de due diligence des briques sélectionnées.	63
I.3 Statut des solutions dans le travail réalisé.	64

Glossaire

API interface de programmation applicative.

DOI Digital Object Identifier.

DORA Digital Operational Resilience Act.

ES expected shortfall.

ETF fonds indiciel coté.

HFT trading haute fréquence.

HITL human-in-the-loop.

IA intelligence artificielle.

IT Information Technology.

JSON JavaScript Object Notation.

LLM grand modèle de langage.

MAR Market Abuse Regulation.

MiFID II Markets in Financial Instruments Directive II.

NIST National Institute of Standards and Technology.

PnL profit and loss.

RAG génération augmentée par récupération.

RL apprentissage par renforcement.

RMF Risk Management Framework.

SDK Software Development Kit.

SQL Structured Query Language.

UE Union européenne.

VaR valeur en risque.

Chapitre 1

Introduction et problématique

1.1 Contexte général

Le trading multifactoriel combine plusieurs expositions pour construire un portefeuille. Les modèles de marché, taille et valeur de [Fama and French \(1993\)](#), puis l'introduction du momentum par [Carhart \(1997\)](#), fournissent des repères pour cette démarche. Pour mon étude, j'ai retenu quatre expositions – value, momentum, quality et low volatility – représentées par des ETF. Je voulais partir de portefeuilles observables et consacrer l'expérience au rôle du LLM, plutôt qu'à la reconstruction des facteurs titre par titre.

À partir de ce socle, j'ai cherché à préciser ce que je pouvais confier à un modèle de langage. Les travaux sur ReAct, Toolformer et AutoGen décrivent différentes façons d'associer un LLM à des outils ou à d'autres agents ([Yao et al., 2023](#) ; [Schick et al., 2023](#) ; [Wu et al., 2023](#)). Dans la première expérience, j'ai limité son rôle à la préparation d'une note à partir des sorties du backtest. Une extension évalue ensuite ses propositions de poids ; les contrôles, les rendements et les coûts restent calculés par Python.

J'ai choisi ce périmètre afin de comparer deux restitutions construites à partir des mêmes résultats quantitatifs : une note déterministe et une synthèse DeepSeek. Mon objectif était de suivre les valeurs reprises par le modèle et de repérer les erreurs que les contrôles laissent passer. J'ai ensuite étendu la comparaison au PnL d'un portefeuille issu de ses propositions, face à deux règles simples.

1.2 Problématique

Ma question de travail est la suivante : comment intégrer un LLM à une chaîne multifactorielle tout en gardant la possibilité de vérifier ses chiffres, ses sources et les décisions qui en découlent ? La synthèse fournit un premier cas de contrôle ; l'allocation simulée engage ensuite les poids du portefeuille et leur effet sur le PnL.

Dans mon prototype, j'examine les valeurs du JSON, les champs cités et la recommandation rédigée. Conserver les bons chiffres n'empêche pas une réponse d'omettre une alerte ou de formuler un avis incompatible avec les permissions. J'évalue donc aussi le validateur, en recherchant les erreurs qu'il accepte.

La revue de MiFID II, de DORA et des cadres NIST et AI Act éclaire les questions de contrôle et de documentation abordées dans ce mémoire ([European Securities and Markets Authority, 2026](#) ; [European Parliament and Council of the European Union, 2022](#) ; [Tabassi, 2023](#) ; [European Parliament and Council of the European Union, 2024](#)). Elle est détaillée au chapitre 3. J'en ai retenu une séparation des responsabilités : le LLM prépare la synthèse ou propose des poids ; le code conserve les calculs et les contrôles avant toute transaction simulée. Le modèle de synthèse n'a pas accès à la modification des poids.

1.3 Objectifs du rapport

J'ai organisé ce rapport autour de trois objectifs :

1. décrire un état de l'art ciblé des briques d'orchestration et de contrôle utiles à ce cas d'usage ;
2. construire un socle ETF déterministe, reproductible et suffisamment simple pour isoler le rôle de la couche agentique ;
3. évaluer les sorties DeepSeek : conformité d'une synthèse, puis admissibilité et performance de propositions d'allocation simulées.

J'ai ainsi construit un cas d'étude dont les sorties peuvent être réexaminées : backtest ETF, notes, propositions d'allocation, rejets et coûts. L'évaluation porte sur ces artefacts ; le gain de compréhension ou de temps pour un analyste demanderait une étude auprès d'utilisateurs.

1.4 Périmètre

Le périmètre empirique est limité à quatre ETF factoriels américains et à SPY. Les ETF servent de proxys investissables des expositions value, momentum, quality et low volatility ; les facteurs ne sont pas reconstruits titre par titre. Les futures, les crypto-actifs et les univers d'actions individuelles sont évoqués dans l'état de l'art, mais ne sont pas expérimentés.

J'ai limité le travail à une démonstration expérimentale de la séparation entre calcul, interprétation et contrôle des sorties LLM. Il ne vise ni une plateforme de production ni la démonstration d'un alpha, et ses résultats ne sont pas une recommandation d'investissement. Les programmes n'envoient aucun ordre réel ; leur validation automatique ne documente pas l'approbation humaine qu'exigerait une utilisation réelle.

L'analyse distingue deux niveaux :

Socle quantitatif déterministe : données, rendements, portefeuille, coûts, risque et diagnostics.

Orchestration et gouvernance LLM : workflow, contrat JSON, permissions, rejet et journalisation.

1.5 Organisation du rapport

La revue des solutions et la méthodologie expliquent les choix qui précèdent l'architecture. Le chapitre 5 rassemble les résultats, depuis la comparaison des notes jusqu'au PnL d'allocation, avec leurs coûts et leurs biais. Je termine par les enseignements que j'en retiens et deux suites prioritaires.

Chapitre 2

État de l’art et solutions GitHub

2.1 Définir l’approche agentique

Dans ce mémoire, le terme « agentique » désigne l’organisation de tâches autour d’un état partagé, d’outils et de règles de passage entre étapes. Le prototype initial en retient une forme limitée : LangGraph coordonne les contrôles et les traitements déterministes, puis peut déclencher une synthèse DeepSeek. Il ne s’agit pas d’une équipe autonome d’agents financiers.

J’ai retenu de ReAct et de Toolformer l’usage d’outils externes pour fournir au modèle les résultats dont il a besoin ([Yao et al., 2023](#) ; [Schick et al., 2023](#)). Dans mon cas d’usage, les rendements, les coûts et les autres valeurs financières sont calculés en amont, puis transmis au modèle. Le RAG pourrait compléter cette architecture pour traiter des documents ou des actualités financières ([Lewis et al., 2020](#)), mais il n’est pas utilisé dans l’expérience.

Reflexion et AutoGen apportent des pistes pour l’architecture cible, avec le retour critique ou les échanges entre agents ([Shinn et al., 2023](#) ; [Wu et al., 2023](#)). Ces mécanismes dépassent le besoin de l’expérience initiale de synthèse : le LLM y restitue un état quantitatif déjà produit. Il ne calcule pas le signal, dont la robustesse doit être évaluée séparément de la qualité de la synthèse.

2.2 État des lieux synthétique du domaine

La comparaison de LangGraph et de Lean m’a aidé à préciser ce que je cherchais. Le premier sert à organiser les étapes d’un workflow ; le second fournit un moteur de backtesting et de simulation. Les traiter comme des solutions concurrentes aurait masqué le besoin de relier orchestration et calculs ([LangChain AI, 2026](#) ; [QuantConnect, 2026](#)).

Le découpage de TradingAgents entre analystes, trader et risque se rapproche de l’organisation que j’envisage pour le desk ([Tauric Research, 2026](#)). C’est cette répartition des rôles que j’ai étudiée, à partir de la documentation. Je n’ai pas exécuté ce dépôt ; les contrôles de mon prototype ont donc été évalués séparément.

J’ai donc retenu une combinaison restreinte : des calculs Python, un workflow LangGraph et une synthèse dont les entrées et les critères d’acceptation sont accessibles. Les tableaux suivants situent ce choix parmi les solutions examinées.

2.3 Trading multifactoriel : chaîne quantitative de référence

Une étude factorielle sur actions individuelles demande de définir l’univers, de préparer les données, de calculer et de normaliser les signaux, puis de construire un portefeuille sous contraintes. Les choix de coûts, de liquidité et de rééquilibrage interviennent dans l’évaluation de la stratégie.

J’ai réduit cette chaîne à quatre ETF, VLUE, MTUM, QUAL et USMV, équipondérés et rééquilibrés chaque mois. Le prototype ne teste donc ni la sélection des titres ni une méthode de combinaison de signaux. Il produit les rendements, les coûts et les diagnostics nécessaires pour examiner ensuite la restitution agentique. Les hypothèses de calcul sont précisées au chapitre 3.

2.4 Frameworks généralistes d’agents

Le tableau 2.1 compare les six frameworks examinés pour organiser les tâches et les échanges entre agents.

TABLE 2.1. Frameworks agentiques généralistes disponibles sur GitHub.

Projet GitHub	Positionnement	Intérêt pour un desk multifactoriel	Limites principales
LangGraph (LangChain AI, 2026)	Workflows d’agents sous forme de graphes, avec état persistant et contrôle fin des transitions.	Adapté aux processus audités : ingestion, calcul de facteurs, revue de risque, validation, journalisation.	Nécessite de développer les outils quantitatifs et les contrôles métier autour du graphe.
AutoGen (Microsoft, 2026a)	Conversations multi-agents et orchestration d’équipes d’agents.	Utile pour comparer des rôles de recherche, analyste, contrôleur de risque et superviseur.	Les conversations libres doivent être fortement contraintes pour un usage régulé.
CrewAI (CrewAI Inc., 2026)	Agents orientés rôles, tâches et processus.	Simple pour construire une équipe d’agents de recherche de marché et de reporting.	Moins adapté seul aux workflows critiques nécessitant des états vérifiables et des garde-fous stricts.
Semantic Kernel (Microsoft, 2026c)	SDK Microsoft pour connecter modèles, plugins, fonctions et mémoire.	Intéressant dans un environnement d’entreprise déjà Microsoft, avec intégration possible à des services internes.	L’architecture de trading reste à construire ; le framework ne fournit pas de modèle financier.
CAMEL (CAMEL-AI, 2026)	Environnements et méthodes pour sociétés multi-agents.	Pertinent pour expérimenter des interactions entre analystes spécialisés.	Plus orienté recherche agentique que production financière.
OpenAI Swarm (OpenAI, 2026)	Exemple éducatif de routage entre agents.	Montre clairement la logique de transfert entre rôles.	Projet expérimental, insuffisant pour une architecture de production.

Pour choisir l’orchestrateur, j’ai privilégié deux besoins du prototype : suivre l’état transmis entre les étapes et arrêter le workflow avant la synthèse si le contrôle qualité échoue. LangGraph répondait à ces besoins, que j’ai vérifiés avec le cas bloqué du protocole. La comparaison avec les autres frameworks est restée documentaire ; elle ne mesure pas leurs performances d’exécution.

2.5 Plateformes quantitatives et briques de trading ouvertes

Le tableau 2.2 rassemble les briques de données, de recherche et de simulation étudiées. Elles couvrent des besoins que l’orchestration LangGraph ne prend pas en charge.

TABLE 2.2. Briques quantitatives GitHub utiles pour une étude multifactorielle.

Projet GitHub	Fonction principale	Apport dans l’architecture cible	Point d’attention
Qlib (Microsoft, 2026b)	Plateforme Microsoft orientée recherche quantitative, données, modèles et backtesting.	Bon socle pour tester des facteurs, pipelines de données et modèles prédictifs.	Adaptation nécessaire aux données internes et aux contraintes de portefeuille réelles.
FinRL (AI4Finance Foundation, 2026b) et FinRL-Meta (AI4Finance Foundation, 2026c)	Apprentissage par renforcement pour trading automatisé et environnements de marché.	Utile pour comparer une approche agentique LLM avec des agents RL plus classiques.	Le RL financier souffre fortement du surapprentissage et des coûts de transaction mal modélisés.
FinRL-X (Yang et al., 2026)	Infrastructure modulaire récente pour trading quantitatif natif IA.	Intéressant comme direction d’évolution vers une infrastructure plus intégrée.	Projet récent : maturité opérationnelle à vérifier avant usage critique.
QuantConnect Lean (QuantConnect, 2026)	Moteur open source de backtesting et trading algorithmique multi-actifs.	Brique robuste pour simuler et exécuter des stratégies dans un cadre contrôlé.	L’agent doit rester au-dessus du moteur, sans contourner les règles d’exécution.
OpenBB (OpenBB Finance, 2026)	Plateforme ouverte d’accès aux données et analyses financières.	Source pratique pour créer un agent de recherche et de veille.	La qualité dépend des fournisseurs de données configurés et de leurs licences.
Freqtrade (Freqtrade, 2026)	Bot de trading crypto avec backtesting et hyperoptimisation.	Utile pour une étude sur crypto-actifs, où l’accès aux données et échanges est simple.	Domaine crypto moins représentatif des contraintes actions institutionnelles.

Dans mon architecture, Qlib ou Lean répondraient à un besoin de recherche et de simulation quantitative, tandis qu’OpenBB interviendrait sur l’accès aux données. Pour le backtest de quatre ETF, j’ai conservé des fonctions Python dont je pouvais suivre les calculs et transmettre directement les sorties au workflow.

2.6 Fiches techniques des briques étudiées

Ces fiches précisent les décisions d’intégration, en complément des tableaux.

2.6.1 LangGraph : orchestration et état

Dans le graphe réalisé, l’état passe du chargement des fichiers aux contrôles qualité et risque, puis à la production de la note. Mon usage de LangGraph porte sur ces transitions et sur le suivi de l’état ([LangChain AI, 2026](#)). Les calculs et les règles métier sont des fonctions Python séparées.

2.6.2 Qlib : recherche quantitative et backtesting

Qlib aurait répondu à un besoin de comparaison de modèles prédictifs ou de pipelines de facteurs ([Microsoft, 2026b](#)). Mon expérience partait de quatre ETF, sans apprentissage de modèle ni reconstruction titre par titre : j’ai donc conservé un moteur Python plus simple. Une recherche sur actions individuelles avec des données point-in-time donnerait davantage de place aux fonctions de cette plateforme.

2.6.3 OpenBB : accès aux données et veille

C’est pour l’agent de veille proposé que j’ai étudié OpenBB ([OpenBB Finance, 2026](#)). Le backtest disposait déjà de son historique ; je n’avais pas besoin d’un second accès aux données pour évaluer les notes. L’intégration d’actualités poserait d’autres questions, notamment leur disponibilité à la date de décision, leur provenance et leur licence.

2.7 Projets GitHub spécifiquement agentiques en finance

Le tableau [2.3](#) présente les cinq projets financiers examinés. Leur intérêt pour l’étude tient surtout aux tâches confiées aux agents et à la manière d’organiser leurs échanges.

TABLE 2.3. Solutions GitHub orientées agents financiers.

Projet GitHub	Principe	Ce qu’il apporte au sujet	Limite pour un desk régulé
TradingAgents (Tauric Research, 2026)	Framework multi-agents imitant une société de trading avec analystes spécialisés, trader et gestion du risque.	C’est l’un des dépôts les plus proches de l’idée d’équipe multi-agents appliquée au trading.	À analyser comme démonstrateur de recherche ; il faut ajouter validation des données, limites de risque, audit et contrôle humain.
FinRobot (AI4Finance Foundation, 2026d)	Plateforme d’agents LLM pour applications financières, notamment recherche actions, analyse et valorisation.	Utile pour automatiser des tâches d’analyste financier et produire des synthèses structurées.	Plus orienté analyse et reporting que génération robuste d’ordres multifactoriels.
FinMem (Yu et al., 2023 ; FinMem contributors, 2026)	Agent de trading fondé sur mémoire stratifiée et profils de décision.	Montre l’intérêt de la mémoire long terme pour contextualiser les décisions de trading.	Les performances expérimentales ne suffisent pas à prouver une robustesse exploitable en production.
AI Hedge Fund (Patel, 2026)	Démonstrateur open source d’un fonds simulé composé d’agents d’investissement.	Bon support pédagogique pour comprendre la division des rôles entre analystes et gestionnaire.	Le dépôt se présente comme preuve de concept, pas comme outil de trading réel.
FinGPT (Yang et al., 2023 ; AI4Finance Foundation, 2026a)	Écosystème de modèles de langage financiers open source.	Peut servir de base de modèle ou de fine-tuning pour des agents spécialisés finance.	Un modèle financier n’est pas une architecture de décision ; il faut ajouter outils, gouvernance et backtesting.

TradingAgents et FinRobot ont nourri deux aspects du projet : la séparation des rôles d’analyse et de risque pour le premier, les tâches de reporting proches de ma note pour le second. Avec FinMem, j’ai surtout retenu la question de la mémoire des synthèses. La conserver introduirait un nouvel objet à évaluer : les informations utiles d’une note passée, mais aussi les erreurs qu’elle transmettrait à la suivante. Je n’ai pas ajouté cette mémoire au prototype.

Ces apports restent documentaires. Aucun des cinq projets n’a été exécuté dans l’expérience ; leurs performances ne sont donc pas comparées à celles du backtest.

2.8 Critères observables et statut dans le mémoire

Les tableaux 2.3 et 2.2 décrivent le positionnement général des outils. Pour rendre la comparaison vérifiable, l’analyse retient aussi des éléments observables : gestion de l’état et persistance, supervision humaine, permissions d’outils, reprise après incident, mémoire,

backtesting, traçabilité, documentation et tests. Le statut de chaque brique est explicité afin de distinguer ce qui a été exécuté de ce qui a seulement été étudié.

TABLE 2.4. Critères observables et statut des principales solutions.

Brique	Éléments observables	Statut	Rôle dans le mémoire
LangGraph	État, transitions, persistance, arrêts, supervision humaine, permissions et trace des sorties.	Utilisée dans le prototype	Orchestration de la démonstration agentique et génération des notes.
Qlib	Pipeline de données, recherche factorielle, backtesting, reproductibilité des expériences.	Référence étudiée	Comparaison du socle quantitatif ; non utilisé dans le calcul final.
Lean	Backtesting multi-actifs, contraintes, simulation de l’exécution et extensibilité.	Référence comparative	Option d’évolution pour un moteur de simulation plus complet.
OpenBB	Accès à des fournisseurs, provenance des données, disponibilité historique et licences.	Référence étudiée	Exemple de brique pour un agent de veille ; non branchée au prototype.
TradingAgents et Fin-Robot	Découpage des rôles, mémoire, synthèse, appels d’outils et décision multi-agents.	Étudiés, non exécutés	Comparaison des architectures financières agentiques.
FinRL et Freqtrade	Environnements de backtesting, apprentissage ou exécution simulée.	Étudiés, non retenus	Solutions utiles dans d’autres périmètres, mais non nécessaires au prototype ETF.

2.9 Lecture critique de l’état de l’art

Après cette comparaison, j’ai limité l’expérience initiale à une question que je pouvais vérifier avec les artefacts produits : que devient un résultat chiffré lorsqu’il est repris dans une synthèse LLM ? Les tableaux ont aidé à choisir les composants ; ils ne suffisaient pas à répondre à cette question.

J’ai donc associé LangGraph à un backtest simple pour confronter les réponses DeepSeek aux valeurs sources et à une note déterministe. La mémoire et la négociation entre agents sont restées hors du prototype. En concentrant les essais sur les critères d’acceptation, je pouvais relier chaque résultat aux contrôles de mon code. Les propriétés des autres projets restent celles observées dans leurs documents.

2.10 Hiérarchie des sources

J’ai hiérarchisé les sources selon leur rôle dans le mémoire. Les articles évalués par les pairs et leurs versions de travail servent à étayer les concepts et les résultats financiers ; les

textes réglementaires et référentiels officiels fixent les contraintes de gouvernance. Les pré-publications sont utilisées pour les travaux récents en indiquant leur statut. Enfin, les dépôts GitHub et leurs documentations servent à vérifier les fonctionnalités, l'activité, les licences et les possibilités d'intégration, mais ils ne remplacent pas une preuve académique.

Google Scholar, ScienceDirect et JSTOR ont servi au repérage et à l'accès. Lorsque cela était possible, j'ai relié la référence à l'article original, à son DOI ou à une version légale déposée par les auteurs ou une institution.

Chapitre 3

Méthodologie d'analyse

3.1 Démarche documentaire

L'état de l'art a été construit à partir de trois types de sources. Les sources académiques servent à comprendre les concepts : agents LLM, raisonnement avec outils, génération augmentée par récupération, mémoire, collaboration multi-agents et modèles factoriels. Les sources réglementaires servent à fixer les contraintes minimales d'un contexte financier : contrôle du trading algorithmique, résilience opérationnelle, gouvernance du risque IA et auditabilité. Enfin, les dépôts GitHub servent à identifier les solutions réellement disponibles et leur degré de maturité. Les articles et documents effectivement mobilisés sont archivés dans le sous-dossier papers lorsqu'une version légale téléchargeable est disponible. Pour les articles dont le texte intégral de l'éditeur n'est pas ouvert, une version de travail ou une copie déposée par les auteurs est privilégiée ; aucune version non autorisée n'est incluse.

La recherche GitHub a été structurée autour de mots-clés : *LLM agent trading, multi-agent financial trading, financial LLM agent, quantitative trading framework, backtesting engine, FinRL, Qlib, OpenBB, LangGraph, AutoGen* et *CrewAI*. Les dépôts retenus devaient satisfaire au moins l'un des critères suivants :

- fournir une brique d'orchestration agentique ;
- traiter explicitement un cas d'usage financier ;
- fournir une infrastructure utile au trading multifactoriel : données, backtesting, moteur d'exécution simulée, apprentissage par renforcement ou recherche quantitative ;

- rendre le code accessible publiquement pour étude, adaptation ou expérimentation.

Le corpus de comparaison comprend 17 fiches : 6 frameworks généralistes, 6 briques quantitatives et 5 projets spécifiquement financiers. Huit dépôts ont ensuite été retenus pour la due diligence finale, tandis que les autres ont été conservés comme références comparatives ou écartés du prototype. Ce décompte porte sur les fiches étudiées et non sur une mesure exhaustive de tous les dépôts repérés. Les dépôts trop éloignés du sujet, purement marketing, sans code exploitable ou uniquement centrés sur des signaux isolés ont été écartés. Les informations sur GitHub doivent être réévaluées avant toute mise en production, car l'activité, la licence, les dépendances et les mainteneurs peuvent évoluer rapidement.

La hiérarchie documentaire est la suivante : les articles scientifiques pour les affirmations théoriques et empiriques, les textes officiels pour les obligations réglementaires, les prepublications pour les résultats récents, les documentations et dépôts officiels pour les fonctionnalités logicielles, puis les documentations de fournisseurs pour la provenance des données. Les moteurs de recherche académique facilitent le repérage, mais ne constituent pas en eux-mêmes des sources citées.

J'ai complété la revue documentaire par un prototype pour confronter les choix d'architecture à des sorties vérifiables. La revue s'appuie sur les 17 fiches de solutions et les références archivées ; le prototype utilise les séries quotidiennes des quatre ETF et de SPY sur 2015–2025, puis leurs sorties structurées. La grille documentaire sert à comparer les briques étudiées. Les mesures de performance, de risque, de turnover et de conformité portent sur les expériences réalisées.

3.2 Revue réglementaire arrêtée au 30 août 2026

Cette revue est documentaire et ne constitue pas un avis juridique. Elle sert à relier l'architecture proposée aux sources officielles disponibles à la date d'arrêt.

- **MiFID II, article 17** : le trading algorithmique implique des systèmes et contrôles de risque adaptés, des limites, des tests et une capacité à décrire les paramètres et contrôles à l'autorité compétente ([European Securities and Markets Authority, 2026](#)).
- **DORA** : le règlement (UE) 2022/2554 s'applique depuis le 17 janvier 2025 et renforce

la résilience opérationnelle numérique des entités financières ([European Parliament and Council of the European Union, 2022](#)).

- **AI Act** : le règlement (UE) 2024/1689 s'applique en principe depuis le 2 août 2026, avec des dates spécifiques pour certaines dispositions, notamment le 2 février 2025, le 2 août 2025 et le 2 août 2027 ([European Parliament and Council of the European Union, 2024](#)). L'applicabilité à un système donné dépend de son usage et de son rôle juridique.
- **NIST AI RMF** : le référentiel AI RMF 1.0 fournit une base de gouvernance, de cartographie, de mesure et de gestion des risques ; la page officielle indique qu'une révision est en cours ([Tabassi, 2023](#)).

De cette lecture, je retiens surtout qu'un passage en production demanderait une validation des fonctions risque, conformité, juridique et IT, en complément du travail technique. Les textes seraient à réexaminer à chaque changement de périmètre, de modèle, de fournisseur ou de régime réglementaire.

3.3 Grille d'évaluation et scoring

La grille compare les solutions sans confondre popularité open source et aptitude à un environnement de marché régulé. La grille détaillée figure dans le tableau [3.1](#). Chaque critère reçoit une note de 0 à 3 :

- 0 : critère absent ou impossible à vérifier dans les sources publiques ;
- 1 : élément partiel, seulement illustré ou nécessitant un développement important ;
- 2 : capacité documentée et utilisable avec une adaptation maîtrisée ;
- 3 : capacité explicite, testable et directement intégrable dans un workflow contrôlé.

TABLE 3.1. Grille de scoring des solutions agentiques et quantitatives.

Critère	Question d'évaluation	Barème 0-3	Interprétation
Couverture fonctionnelle	Le projet couvre-t-il orchestration, données, backtest, risque, exécution ou reporting ?	0 absent ; 1 partiel ; 2 plusieurs fonctions ; 3 chaîne cohérente	Mesure l'étendue, sans assimiler étendue et maturité.
Reproductibilité	Les calculs et décisions peuvent-ils être rejoués avec les mêmes données et paramètres ?	0 non vérifiable ; 1 exemples ; 2 pipeline documenté ; 3 tests et versionnement	Condition de base pour audit et validation.
Explicabilité	Les sorties relient-elles la justification aux signaux, sources et contraintes ?	0 aucune ; 1 texte libre ; 2 sorties structurées ; 3 preuve liée aux données	Distingue une narration plausible d'une explication vérifiable.
Contrôle du risque	Existe-t-il des limites, validations, stress tests ou garde-fous ?	0 absent ; 1 implicite ; 2 documenté ; 3 bloquant et testé	Critère critique avant toute exécution.
Qualité logicielle	Le dépôt propose-t-il documentation, tests, exemples et structure maintenable ?	0 absent ; 1 incomplet ; 2 présent ; 3 intégré et suivi	Réduit le risque opérationnel.
Intégrabilité	Le projet s'interface-t-il avec Python, bases, API ou moteurs de backtesting ?	0 isolé ; 1 adaptation forte ; 2 interfaces disponibles ; 3 intégration directe	Détermine la faisabilité d'une architecture composée.
Gouvernance	Les rôles, permissions, journaux et validations sont-ils explicitement modélisés ?	0 absent ; 1 déclaratif ; 2 partiel ; 3 contrôlable et auditable	Indispensable dans une institution financière.

J'ai utilisé le barème sur 21 points pour expliciter mes arbitrages entre les dépôts, présentés dans le tableau 3.2. Ces notes sont des appréciations documentaires provisoires, sans valeur de certification ou de validation de production.

TABLE 3.2. Scoring documentaire provisoire des dépôts retenus.

Dépôt	Couverture	Reprod.	Explic.	Risque	Qualité	Intégr.	Gouv.	Total
LangGraph	2	3	2	1	3	3	2	16/21
Qlib	3	3	1	1	3	3	1	15/21
Lean	3	3	1	2	3	3	1	16/21
OpenBB	2	2	1	0	3	3	1	12/21
TradingAgents	2	1	2	0	2	2	1	10/21
FinRobot	2	1	2	0	2	2	1	10/21
FinRL	2	2	1	1	2	2	1	11/21
Freqtrade	3	2	1	1	3	3	1	14/21

Le tableau confirme le choix d'une architecture composée. LangGraph obtient son meilleur niveau sur l'orchestration et l'intégrabilité, tandis que Qlib et Lean sont plus solides

pour la recherche et le backtesting. Les scores de gouvernance et de contrôle du risque restent faibles ou partiels, ce qui justifie l'ajout de règles métier externes et d'une validation humaine.

3.4 Principes de conception

Ces principes ne sont pas des hypothèses statistiques. Ils fixent les règles de conception qui relient le socle quantitatif, la couche agentique et la gouvernance :

Séparation du calcul et de l'interprétation : les rendements, coûts et indicateurs de risque sont calculés par des fonctions déterministes. Dans la synthèse, les pondérations sont également fixées par le moteur. Dans l'extension d'allocation, le modèle propose des poids ; le code les valide puis calcule les positions et leur PnL.

Supervision humaine : une recommandation agentique est un objet de revue. Toute action engageant un portefeuille réel, et notamment l'envoi d'un ordre, reste soumise à une validation explicite et à des permissions indépendantes.

Explicabilité vérifiable : une justification doit renvoyer aux données, aux versions, aux contraintes et aux règles réellement utilisées. Le texte produit ne remplace pas le journal structuré.

3.5 Méthode de classement des solutions

Chaque projet est classé selon son rôle principal :

Orchestration : organisation des agents, des tâches, de l'état et des outils.

Recherche financière : collecte, résumé et analyse d'informations financières.

Infrastructure quantitative : calcul de signaux, backtesting, environnement de simulation ou moteur d'exécution.

Gouvernance : capacité à tracer, valider, surveiller et expliquer les décisions.

Cette classification évite une erreur fréquente : comparer un framework agentique comme LangGraph à un moteur de backtesting comme Lean. Les deux sont complémentaires. Dans

une architecture cible, LangGraph peut piloter le workflow, Qlib calculer des signaux, OpenBB fournir des données, Lean simuler l'exécution, et un agent spécialisé produire un rapport de décision.

3.6 Protocole expérimental initial

J'ai retenu un prototype quantitatif minimal afin de tester le socle déterministe de l'architecture proposée. Cette simplicité est volontaire : elle permet de concentrer l'expérience sur la reproductibilité des données, des calculs de portefeuille, des coûts et des indicateurs de risque avant toute intervention d'un agent de synthèse. Le prototype ne cherche donc pas à démontrer qu'un agent ou une stratégie produit systématiquement de l'alpha.

Le protocole utilise quatre ETF factoriels américains comme proxies investissables de sleeves de valeur, momentum, qualité et faible volatilité : VLUE, MTUM, QUAL et USMV. Ces ETF ne constituent pas une reconstruction titre par titre des facteurs value, momentum, quality et low volatility : ils servent de portefeuilles observables représentant ces expositions. Le benchmark est SPY. Les cours ajustés sont téléchargés via `yfinance` (Aroussi, 2026) sur la période allant du 1er janvier 2015 au 31 décembre 2025. Le portefeuille multifactoriel est équipondéré et rééquilibré à la fin de chaque mois. Les prix de fin de mois représentent un prix d'exécution théorique ; aucune exécution au jour suivant, bid-ask spread ou impact de marché n'est simulé. Le portefeuille est supposé déjà investi aux poids cibles à la première date, de sorte que le coût initial n'est pas inclus.

Trois hypothèses de coûts de transaction sont testées : 0, 10 et 30 points de base par unité de turnover. Le taux sans risque est fixé à 0 % pour faciliter la comparaison du ratio de Sharpe. Les indicateurs calculés sont le rendement annualisé, la volatilité annualisée, le ratio de Sharpe, le maximum drawdown, la valeur finale d'un investissement initial de 100 unités et le turnover.

Pour un mois t , le rendement net du portefeuille est calculé selon :

$$r_t^{\text{net}} = \sum_i w_{i,t-1} r_{i,t} - c \tau_t, \quad \tau_t = \frac{1}{2} \sum_i \left| w_{i,t}^{\text{cible}} - w_{i,t}^{\text{aprèsrendement}} \right|,$$

où c désigne le coût unitaire exprimé en fraction et τ_t le turnover au rebalancement. Sur T

rendements mensuels, les métriques principales sont :

$$R_{\text{ann}} = \left(\prod_{t=1}^T (1 + r_t) \right)^{12/T} - 1, \quad \sigma_{\text{ann}} = \sqrt{12} \text{sd}(r_t), \quad S = \sqrt{12} \frac{\text{mean}(r_t)}{\text{sd}(r_t)}.$$

Le turnover est défini en convention « one-way » : il représente la valeur des achats (égale à celle des ventes dans un rééquilibrage neutre), et les coûts sont calculés comme $c\tau_t$. Le turnover annualisé est calculé par $12 \sum_t \tau_t / T$. Le drawdown est obtenu à partir du rapport entre la valeur courante et le maximum historique de la valeur cumulée. Ces formules rendent explicite le traitement des coûts, du turnover et de l'annualisation.

Le déroulement peut être résumé par l'algorithme 1.

Algorithme 1 Calcul du portefeuille multifactoriel et des métriques

Entrées : Cours ajustés des quatre ETF factoriels et de SPY, période d'étude, coûts $\mathcal{C} = \{0, 10, 30\}$

pb

Sorties : Rapports de qualité, métriques, diagnostics de rééquilibrage et figures

- 1 : Télécharger les cours et vérifier les dates, valeurs manquantes et prix non positifs
 - 2 : Supprimer les dates incomplètes et enregistrer `quality_report.json`
 - 3 : Calculer les rendements mensuels et initialiser les poids cibles à 25 % par sleeve
 - 4 : **pour** chaque coût $c \in \mathcal{C}$ **faire**
 - 5 : Réinitialiser le portefeuille aux poids cibles
 - 6 : **pour** chaque mois t **faire**
 - 7 : Calculer le rendement brut et les poids après rendement
 - 8 : $\tau_t \leftarrow \frac{1}{2} \sum_i |w_{i,t}^{\text{cible}} - w_{i,t}^{\text{après}}|$
 - 9 : $r_t^{\text{net}} \leftarrow r_t^{\text{brut}} - c\tau_t$
 - 10 : Enregistrer rendement, turnover, coût et poids
 - 11 : **fin pour**
 - 12 : Calculer rendement annualisé, volatilité, Sharpe, drawdown et valeur finale
 - 13 : Exporter les diagnostics de rééquilibrage pour le coût c
 - 14 : **fin pour**
 - 15 : Comparer le benchmark et le multifactoriel sur l'échantillon complet
 - 16 : Répéter la comparaison sur la période hors échantillon 2021–2025
 - 17 : Exporter `metrics.csv`, `period_metrics.csv` et les figures
-

Le calcul sépare l'échantillon complet de la période hors échantillon allant de janvier 2021

à décembre 2025. Comme aucun paramètre n'est appris sur les rendements, cette coupure constitue un contrôle de robustesse temporelle et non une validation complète d'un modèle prédictif. Les métriques des deux périodes sont conservées dans `period_metrics.csv`.

Le déroulement du protocole est résumé par la figure 3.1. Le téléchargement contient 2 766 observations quotidiennes. Le contrôle qualité n'a relevé ni date dupliquée, ni valeur manquante, ni prix non positif. L'écart calendaire maximal est de quatre jours, aucun gap supérieur à sept jours n'est observé et aucun mouvement quotidien supérieur à 25 % n'est signalé. Les contrôles et leurs limites connues sont archivés dans `data/quality_report.json`. Les cours sont auto-ajustés par la source, mais les corporate actions, le survivorship bias, la liquidité et le slippage nécessitent encore une validation institutionnelle ; les scénarios de 10 et 30 points de base restent des proxys de coûts tout compris.

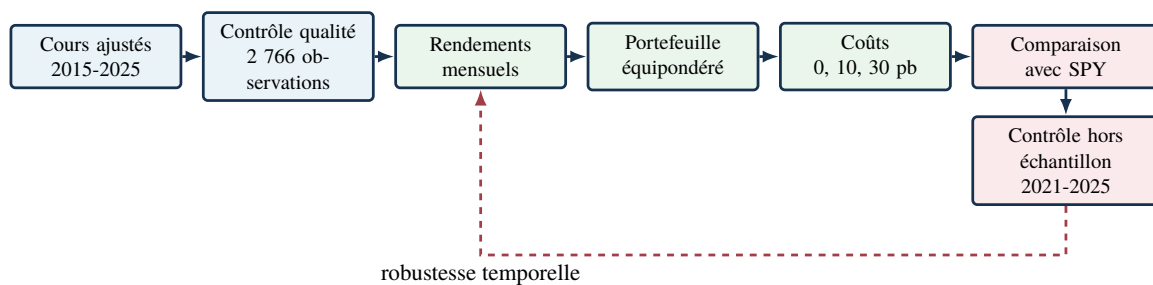


FIGURE 3.1. Déroulement du protocole expérimental et contrôle hors échantillon.

3.7 Protocole d'évaluation de la couche LLM

L'évaluation de la couche LLM complète le benchmark quantitatif sans lui être substituée. Elle ne cherche pas à montrer qu'un modèle de langage crée de l'alpha ; elle vérifie plutôt qu'un agent peut lire un état quantitatif déjà calculé, restituer ses champs structurés sans les modifier et respecter les permissions et les alertes produites par le workflow déterministe.

Le protocole est exécuté par `evaluate_deepseek.py`. Pour chaque cas, le script recharge les mêmes artefacts que le système étudié : `data/quality_report.json`, `results/monthly_asset_returns.csv` et `results/rebalance_diagnostics_10bps.csv`. Il reconstruit ensuite l'état à une date donnée en réutilisant les fonctions de chargement, de contrôle qualité, d'analyse du rééquilibrage et de contrôle du risque. L'évaluation porte sur douze cas :

dix dates historiques, un scénario synthétique de turnover élevé entraînant une alerte de risque et un cas où le contrôle qualité bloque explicitement la suite. Dans ce dernier cas, aucun appel API n'est effectué.

Le modèle reçoit exclusivement un objet JSON structuré contenant la date, l'état qualité, les rendements factoriels, le rendement net, le turnover, le coût, les poids cibles, les alertes de risque, les permissions et les contraintes. Il ne reçoit ni les cours bruts, ni un outil de recherche, ni une capacité de placement d'ordre. La sortie attendue est également contrainte par un contrat JSON : résumé, valeurs citées, recommandation descriptive, limites, indicateurs de permission et liste des champs sources cités. Je peux ainsi distinguer la génération linguistique de la décision quantitative.

L'expérience archivée a été réalisée le 7 septembre 2026. Les métadonnées des 33 réponses identifient le modèle `deepseek-v4-flash` et l'API <https://api.deepseek.com>. La configuration enregistrée comporte dix cas historiques, un stress de turnover, un cas bloqué et trois répétitions par cas non bloqué. L'appel défini dans le code utilise une température nulle, une réponse en objet JSON et un plafond de 3 000 jetons de sortie. Ces paramètres d'appel ne sont pas tous enregistrés dans les métadonnées historiques, ce qui limite la reconstitution exacte de la requête.

Le validateur compare les champs numériques aux valeurs sources avec une tolérance absolue de 10^{-4} , vérifie la présence des champs attendus et contrôle les indicateurs booléens de permission. L'évaluateur ajoute des contrôles sur les champs supplémentaires, les noms ou préfixes des champs cités, ainsi que la présence de marqueurs d'alertes. Ces vérifications supplémentaires ne sont pas toutes appliquées par le workflow principal. Les contrôles textuels reposent sur des mots-clés ou sur la mention du champ d'alertes ; ils ne vérifient pas la cohérence sémantique de chaque phrase ni la restitution de chaque alerte.

La stabilité des sorties est mesurée sur les valeurs numériques normalisées à quatre décimales et sur les indicateurs de permission ; une mesure distincte compare l'ensemble des champs cités. Le code d'évaluation réessaie les appels qui lèvent une exception, dans une limite de trois tentatives. Une réponse reçue mais rejetée par le score est comptabilisée comme un échec ; le mécanisme actuel ne la régénère pas automatiquement. Les métadonnées archivées ne fournissent pas un journal exhaustif des éventuelles tentatives intermédiaires.

Les résultats sont enregistrés dans `results/deepseek_evaluation.json` et `results/deepseek_evaluation.csv`. Les métadonnées comprennent le modèle, l'URL de l'API, l'identifiant et l'horodatage de requête, la latence et les nombres de jetons. L'archive du 7 septembre ne contient ni empreinte du prompt, ni empreinte des données, ni commit Git. Le code actuel prévoit ces éléments pour de nouvelles traces du workflow DeepSeek, mais cette extension ne complète pas rétroactivement les preuves de l'expérience. La clé d'API est lue depuis `.env` et n'apparaît pas dans les sorties. Le protocole mesure la conformité aux contrôles disponibles ; il n'évalue pas la performance financière ni la productivité humaine.

3.7.1 Analyse complémentaire des notes et du validateur

Une analyse hors réseau complète l'expérience archivée : elle compare la note déterministe de la dernière date commune avec la première réponse du cas historique correspondant, puis examine les deux autres répétitions. La grille porte sur l'exactitude, la clarté, les alertes, la traçabilité et la longueur ; l'appréciation de lisibilité est qualitative et ne repose pas sur un panel indépendant. Trois altérations portant chacune sur un seul champ et deux témoins de détection sont ensuite appliqués à la première réponse. Ces essais construits évaluent les limites du validateur, sans être assimilés à des erreurs spontanées du modèle. Le script `analyze_note_comparison.py` conserve les entrées, les transformations, les scores et les empreintes des sources, sans appel API ni modification de l'évaluation initiale.

3.8 Extension : décisions d'allocation et PnL simulé

L'expérience de synthèse ne peut pas modifier le PnL puisqu'elle reprend une allocation déjà calculée. Pour étudier l'effet d'une décision du modèle, j'ai ajouté une seconde expérience : DeepSeek propose des poids cibles, puis un moteur Python indépendant les accepte ou les rejette et calcule les positions. Cette extension utilise le module `agent_allocation_backtest.py`. Elle n'exécute pas les sept agents de l'architecture cible et ne passe pas par le graphe LangGraph de restitution. Elle teste un rôle d'allocation LLM encadré par des règles déterministes, sans connexion à un courtier.

3.8.1 Période, références et informations disponibles

La collecte a été réalisée le 12 septembre 2026 sur 56 décisions mensuelles, pour les mois de mai 2021 à décembre 2025. Chaque portefeuille débute avec 100 unités, déjà investies à 25 % dans chaque ETF à la clôture du 30 avril 2021. Les anciennes positions supportent la variation jusqu'à la première exécution. Les coûts d'entrée initiale et de liquidation finale sont exclus.

Trois politiques utilisent les mêmes cours ajustés, le même calendrier, les mêmes contraintes et le même moteur de frais : une cible équilibrée de 25 % ; une allocation proportionnelle à l'inverse de la volatilité historique, plafonnée à 40 % ; et les poids proposés par DeepSeek. Pour la règle inverse volatilité, le poids dépassant le plafond est fixé à 40 %, puis le solde est réparti proportionnellement entre les actifs encore libres. La procédure est répétée si nécessaire. Les trois politiques restent soumises au contrôle de turnover, sans ajustement discrétionnaire après un rejet.

À la dernière séance du mois précédent, les entrées comprennent les rendements des quatre ETF sur 21, 63, 126 et 252 séances, la volatilité des 252 derniers rendements quotidiens annualisée avec $\sqrt{252}$, le drawdown sur les 253 derniers cours, les poids courants et les contraintes. Ces horizons sont des approximations en séances de un, trois, six et douze mois. Les prix sont coupés à la date d'observation avant le calcul des indicateurs. Le modèle ne reçoit ni cours d'exécution futur, ni rendement du mois à venir, ni actualité. Un test modifie les cours futurs et vérifie que les entrées demeurent identiques. Cette propriété du programme ne résout pas le biais de préentraînement discuté en section 5.6.5.

3.8.2 Contrat, contraintes et traitement d'un rejet

La sortie doit contenir exactement quatre champs : `decision_date`, `target_weights`, `reason` et `cited_fields`. La date doit correspondre à celle des entrées. Les quatre ETF sont obligatoires ; les poids doivent être des nombres finis, non négatifs, dont la somme vaut 1, sans levier ni poche de liquidités. La concentration maximale est de 40 % par poids cible. Une tolérance de 10^{-9} est admise sur la somme ; la normalisation dans cette seule tolérance est enregistrée. Les poids effectivement appliqués figurent dans le journal.

Le turnover proposé est calculé par $\frac{1}{2} \sum_i |w_i^* - w_i^{\text{courant}}|$ et limité à 10 %. Les chemins cités doivent exister exactement dans l'objet transmis, et la justification doit être un texte non vide. Ce validateur d'allocation est distinct de celui des 33 synthèses : il contrôle les contraintes numériques et les chemins complets, mais pas la pertinence économique du raisonnement ni la véracité de chaque phrase.

Une proposition rejetée conserve les *quantités* déjà détenues, sans facturer de transaction. Les poids continuent alors à varier avec les prix ; ils ne sont pas réinitialisés à 25 %. Le moteur ne corrige pas les poids hors limites et ne demande pas une autre réponse pour remplacer le rejet. Le plafond de concentration s'applique aux poids cibles, sans garantir que les mouvements de marché le respectent entre deux rééquilibrages. Le choix de conserver les positions fait partie de la politique évaluée.

3.8.3 Exécution différée et frais autofinancés

La proposition est produite à partir de la clôture du mois précédent ; l'exécution simulée intervient à la première clôture disponible du mois suivant. Le turnover est revérifié à ce cours, avant puis après prise en compte des frais. Un dépassement provoque également la conservation des positions. Une proposition admissible à l'observation peut ainsi être inexécutable sous les contraintes retenues.

Soient q_i les quantités avant transaction, P_i les cours ajustés d'exécution, $N_i = q_i P_i$ et $V = \sum_i N_i$. Avec les poids cibles w_i^* et le taux $c = 0,001$, soit 10 points de base, les frais F satisfont :

$$F = \frac{c}{2} \sum_i |w_i^*(V - F) - N_i|, \quad q'_i = \frac{w_i^*(V - F)}{P_i}. \quad (3.1)$$

Le moteur résout cette équation par dichotomie. Le turnover réellement échangé vaut $\frac{1}{2V} \sum_i |q'_i P_i - N_i|$. La valeur des nouvelles positions et les frais doivent vérifier $\sum_i q'_i P_i + F = V$. Les frais sont donc financés par le portefeuille et portent sur la moitié du volume effectivement échangé. Avec ces frais, les montants achetés et vendus ne sont plus exactement égaux.

Les unités fractionnaires obtenues à partir de cours ajustés représentent une comptabilité de rendement total. Elles ne constituent pas des quantités directement utilisables auprès d'un

courtier. Aucune simulation séparée du spread, de l’impact de marché, de la liquidité ou de la fiscalité n’est réalisée. Le taux de 10 points de base est une hypothèse expérimentale, sans calibrage empirique sur des transactions réelles.

Les valeurs sont suivies chaque séance. Pour D jours calendaires entre le capital initial et la dernière valorisation, le rendement annualisé est $(V_T/V_0)^{365,25/D} - 1$. La volatilité et le Sharpe utilisent les rendements quotidiens avec facteur $\sqrt{252}$ et taux sans risque nul ; le drawdown inclut la valeur initiale. Ces conventions diffèrent du backtest initial : période plus courte, exécution différée, frais autofinancés et mesures de risque quotidiennes. Les chiffres des deux expériences ne doivent donc pas être comparés comme s’il s’agissait d’un même protocole. Aucun nouveau scénario LLM à 0 ou 30 points de base n’a été collecté ; ces taux servent uniquement aux tests comptables de cette extension.

3.8.4 Configuration du modèle et budget de collecte

Le modèle demandé est `deepseek-v4-flash` ; les 56 réponses retournent l’identifiant `deepseek-flash`. Les deux noms sont archivés, sans possibilité de vérifier la version exacte des poids internes du fournisseur. La campagne utilise une température nulle, le mode JSON, un plafond de 3 000 jetons de sortie et le réglage explicite `thinking: disabled`, documenté par DeepSeek ([DeepSeek, 2026](#)). La température ne suffit pas à garantir des réponses identiques lors de futurs appels.

La période avait d’abord été envisagée sur les 60 mois de 2021–2025. Quatre tentatives techniques ont précédé la collecte : trois réponses sans JSON ont atteint le plafond de 3 000 jetons de raisonnement, puis une quatrième requête a été interrompue. Après désactivation de ce mode, les 56 appels restants ont été consacrés à une période commune de mai 2021 à décembre 2025. Ce changement répond à la limite de 60 tentatives et a été décidé avant examen des performances de la campagne corrigée. Les quatre tentatives initiales sont conservées séparément et exclues de ses décisions ; elles restent incluses dans le bilan des ressources consommées.

Il n’y a qu’une réponse par date, sans sélection entre répétitions, ni nouvelle génération après rejet. Chaque tentative est écrite sur disque avant l’appel HTTP et consomme une place du budget, même en cas d’erreur ou d’interruption. Une erreur API arrête la collecte ; une

reprise conserve cette tentative comme une absence de proposition. Le protocole ne mesure donc pas la variabilité du PnL entre plusieurs générations.

3.8.5 Journalisation et reproduction

L’archive contient le prompt, les indicateurs, les poids courants, les paramètres, leurs empreintes SHA-256, la réponse brute, les identifiants de requête et de modèle, le statut de fin, les jetons rapportés et la durée. Le journal d’exécution ajoute les quantités avant et après décision, les poids appliqués, les frais, le turnover et les motifs de rejet. Les empreintes du fichier de prix, du code et de l’archive sont également enregistrées.

Le mode de reproduction hors réseau vérifie la correspondance exacte de la requête avant de réutiliser la réponse. Modifier les frais, les poids courants ou le prompt invalide cette correspondance. Il s’agit d’une reproductibilité des calculs à décisions archivées, pas d’une garantie de stabilité du service LLM. Les commandes et fichiers nécessaires figurent en annexe III. Un test technique sans LLM utilise une règle de momentum sur 60 mois ; ses performances ne sont pas assimilées à celles de DeepSeek.

3.9 Limites de la méthode

Le benchmark initial évalue un socle quantitatif simple et sa synthèse contrôlée. L’extension mesure le PnL de propositions d’allocation validées, sur une seule trajectoire rétrospective. Aucun des deux dispositifs ne valide un système agentique complet. Un univers plus large, des données historiques point-in-time et une modélisation de la liquidité seraient nécessaires pour étendre la portée financière des résultats.

Les fonctionnalités, licences et conditions de maintenance des dépôts GitHub peuvent évoluer. Leur adoption demande une revue technique datée, une analyse de sécurité des dépendances et une validation juridique de la licence.

Chapitre 4

Architecture cible pour un desk multifactoriel agentique

4.1 Principe directeur

J’ai retenu une règle : toute métrique financière et toute valeur de portefeuille doivent provenir d’un composant déterministe ; un poids proposé par le LLM reste une décision à valider. Dans le prototype de synthèse, LangGraph organise la lecture des résultats, les contrôles et la production de la note. Le LLM intervient dans la restitution ; les calculs restent rejouables à partir des fichiers sources.

Ce chapitre distingue les interfaces réalisées des fonctions proposées pour un desk plus complet. Le choix de LangGraph ne change pas cette répartition des responsabilités.

4.2 Vue d’ensemble de l’architecture

La figure [4.1](#) situe les données, les calculs, les agents et la gouvernance dans le dispositif proposé. Les flèches suivent le passage de l’état : une sortie non conforme serait renvoyée au composant responsable avant de poursuivre. La cible décrit sept rôles, mais le prototype exécuté est plus restreint : il implémente le chargement des données, les gardes qualité et risque, un superviseur déterministe et un nœud de synthèse DeepSeek. Les agents actualités et conformité restent proposés. Le rôle portefeuille fait l’objet de l’extension distincte décrite

en section 3.8, sans instancier les sept agents de cette architecture.

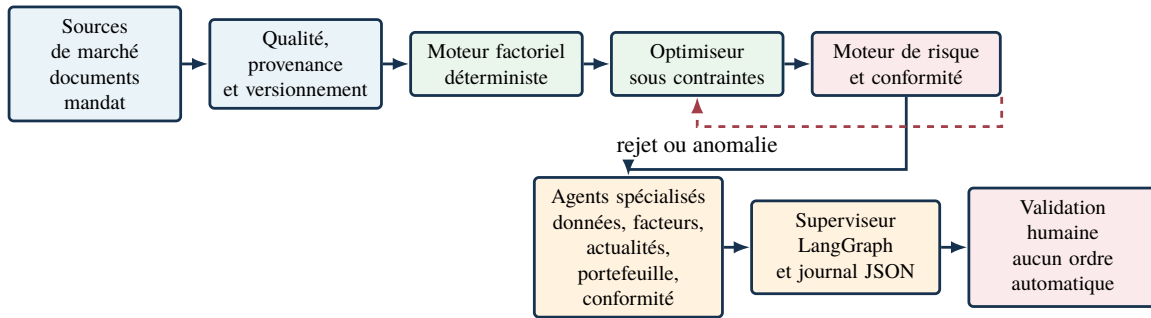


FIGURE 4.1. Architecture cible séparant le socle quantitatif, les agents et la gouvernance.

4.3 Socle quantitatif déterministe

Dans le prototype initial, le moteur déterministe calcule les rendements, les poids après rendement, les coûts, le turnover et les métriques de performance et de risque. La couche agentique lit ces résultats déjà produits. Elle ne dispose ni d'un optimiseur sous contraintes, ni d'un moteur institutionnel de limites ou de liquidité. J'ai ainsi gardé les valeurs sources accessibles pour les comparer à la restitution du modèle.

Dans le workflow exécuté, une anomalie signalée par le rapport qualité bloque l'accès à la synthèse ; les calculs du backtest ont été effectués en amont. Une sortie LLM qui échoue aux contrôles disponibles est ensuite rejetée et archivée avec ce statut. Le seuil de turnover de 10 % a un autre rôle : son dépassement ajoute une alerte d'escalade, sans interrompre la synthèse. Le statut reste « à valider » et aucun outil d'exécution n'est exposé. Le blocage sur limites financières est absent de ce workflow de synthèse ; il est implémenté dans l'extension d'allocation. L'enregistrement d'une décision humaine appartient toujours à l'architecture cible.

Le contrat fonctionnel minimal du socle est le suivant :

- entrée : source de données, univers, horizon, paramètres de marché et contraintes de portefeuille ;
- traitement : calcul déterministe, versionné et rejouable ;
- sortie : valeurs numériques structurées, horodatage, paramètres utilisés, hypothèses et éventuelles erreurs ;

- contrôle : conservation des logs, séparation des droits et capacité à refaire le calcul avec les mêmes données.

4.4 Agents proposés

TABLE 4.1. Rôles, permissions et limites des agents proposés.

Agent	Mission	Outils autorisés	Sortie attendue	Interdictions	Conditions d'escalade
Données	Vérifier disponibilité, fraîcheur, provenance et cohérence.	APIs de données, contrôles qualité, logs.	Rapport de qualité et alertes.	Ne pas corriger silencieusement une donnée.	Donnée manquante, périmée ou incohérente.
Facteurs	Lancer les calculs et comparer les distributions.	Moteur déterministe, notebooks validés.	Scores, variations et anomalies.	Ne pas changer la formule ni les paramètres.	Rupture de distribution ou résultat non rejouable.
Actualités	Résumer nouvelles, résultats et événements macro.	OpenBB, flux autorisés, base RAG.	Synthèse sourcée et confiance.	Ne pas modifier les poids ni produire d'ordre.	Sources contradictoires ou absence de source.
Portefeuille	Proposer une allocation sous contraintes.	Optimiseur, mandat et coûts.	Poids cibles et attribution.	Ne pas contourner un rejet risque ou confort.	Allocation infaisable ou contrainte violée.
Risque	Contrôler expositions, volatilité, drawdown et scénarios.	Moteur de risque, limites, stress tests.	Avis accepté, rejeté ou escaladé.	Ne pas relever lui-même une limite.	Limite franchise, modèle absent ou liquidité inconnue.
Conformité	Vérifier restrictions, documentation et séparation des rôles.	Référentiel de règles, listes, audit.	Validation ou blocage motivé.	Ne pas accorder d'exception implicite.	Règle inconnue, conflit ou trace incomplète.
Superviseur	Agréger les avis et préparer la revue humaine.	État complet, règles de délégation, journal.	Note de décision traçable.	Ne pas exécuter ni masquer un désaccord.	Désaccord, confiance faible ou intervention requise.

Les rôles du tableau 4.1 définissent la cible. Les permissions, représentées dans la figure 4.2, devraient être imposées par les outils accessibles à chaque rôle. Dans le prototype de synthèse, cette séparation se traduit par l'absence d'outil permettant au LLM de modifier les poids. Dans l'extension, il émet une proposition JSON soumise à Python. Aucun des deux dispositifs ne dispose d'un outil d'ordre réel.

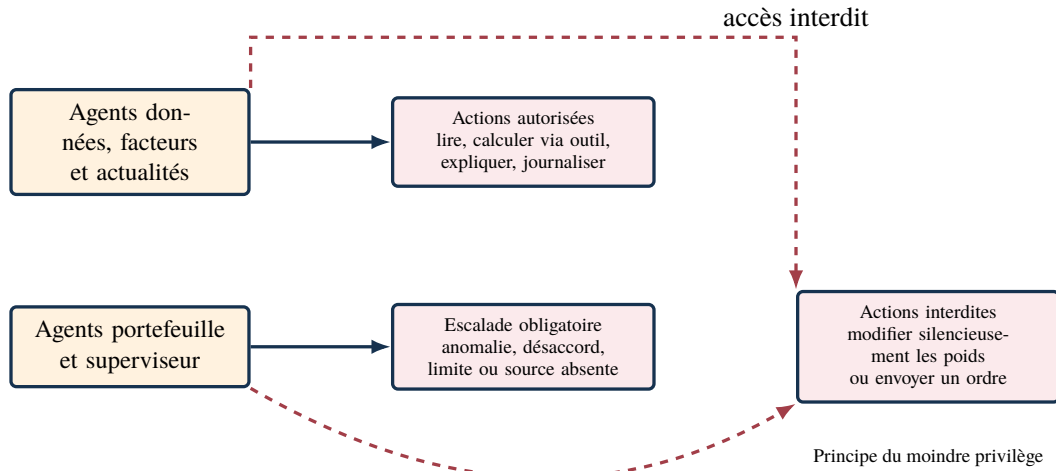


FIGURE 4.2. Permissions, escalades et interdictions dans la couche agentique.

4.5 Flux de décision et boucle de rejet

Le tableau 4.2 et la figure 4.3 décrivent le flux cible et ses conditions de rejet. Une anomalie devrait interrompre le passage à l'étape suivante ou renvoyer l'état au composant responsable.

TABLE 4.2. Séquence de décision et conditions de rejet.

Étape	Émetteur	Destinataire	Échange et condition
1	Données	Contrôle qualité	Les données et leur version sont chargées ; une anomalie bloque le flux.
2	Contrôle qualité	Moteur factoriel	Le calcul démarre seulement si les contrôles sont conformes.
3	Moteur factoriel	Optimiseur	Les signaux et paramètres sont transmis sous forme structurée.
4	Optimiseur	Moteur de risque	Les poids proposés sont contrôlés avec les coûts et les contraintes.
5	Moteur de risque	Optimiseur	En cas de rejet, l'état revient à l'optimiseur avec le motif ; aucune allocation n'est validée.
6	Risque et conformité	Superviseur	Les avis, alertes et permissions sont agrégés sans supprimer les désaccords.
7	Superviseur	Humain puis archive	La validation, la modification ou le rejet sont enregistrés dans le journal JSON.

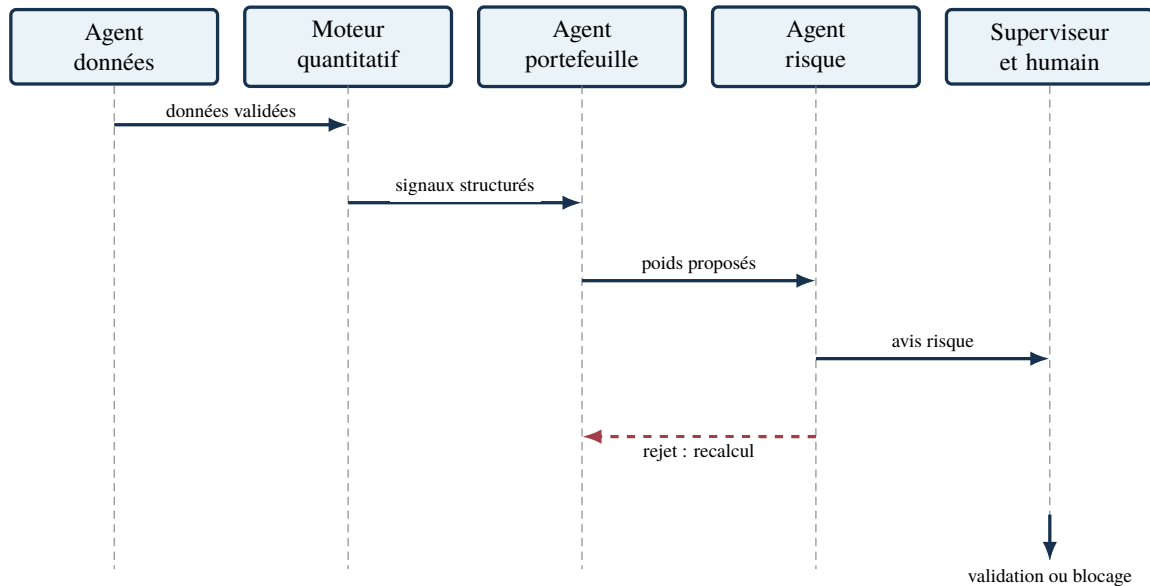


FIGURE 4.3. Séquence de décision avec boucle de rejet par le contrôle risque.

Dans cette cible, une donnée manquante arrêterait le flux avant le calcul factoriel. Le prototype applique un contrôle plus limité : il lit le rapport qualité après le backtest et bloque, si nécessaire, l'accès à la synthèse. La décision humaine enregistrée reste à développer.

4.6 Journal de décision explicable

Le journal proposé pour un desk complet réunirait les versions des données et du code, les avis des agents et la décision humaine pour chaque recommandation. Les traces déterministes archivées contiennent les valeurs quantitatives, les alertes et les permissions. L'évaluation DeepSeek du 7 septembre 2026 ajoute notamment l'identifiant du modèle, l'horodatage et l'identifiant de requête, mais ne contient pas les empreintes du prompt et des données ni le commit Git. Le code actuel prévoit ces informations pour les nouvelles traces du workflow DeepSeek ; elles ne peuvent pas être attribuées rétroactivement à l'expérience archivée. Aucune de ces sorties n'enregistre une validation humaine effectivement réalisée.

L'extrait suivant provient de la trace du 30 novembre 2025 ; il est présenté dans la figure 4.4. Je l'utilise pour vérifier que les rendements, le rendement net, le turnover, les poids cibles et les permissions sont conservés au même endroit.

```

{
  "workflow": "langgraph_deterministic_supervisor",
  "date": "2025-11-30",
  "quality_ok": true,
  "risk_status": "a valider",
  "rebalance": {
    "factor_returns": {
      "Value": 0.0249981345118395,
      "Momentum": -0.0156164272534177,
      "Quality": 0.0090677798205325,
      "Low volatility": 0.0230736302539755
    },
    "net_return": 0.0103740219288315,
    "turnover": 0.0067574044008865,
    "transaction_cost": 6.757404400886513e-06,
    "target_weights": {
      "VLUE": 0.25, "MTUM": 0.25,
      "QUAL": 0.25, "USMV": 0.25
    }
  },
  "permissions": {
    "can_change_weights": false,
    "can_send_orders": false,
    "human_validation_required": true
  }
}

```

FIGURE 4.4. Extrait réel du journal JSON produit par le prototype.

Pour examiner une synthèse, je repars des valeurs et des permissions enregistrées dans cette trace. L'absence d'outil de modification des poids ou d'envoi d'ordres se vérifie dans le code ; les indicateurs JSON en documentent la règle. Le texte libre demande un contrôle séparé de sa cohérence avec ces permissions.

4.7 Monitoring continu

Le tableau 4.3 et la figure 4.5 proposent des métriques et des réactions pour l'architecture cible. Ces seuils ne proviennent pas d'un calibrage empirique du prototype. Leur fixation demanderait des données d'exploitation, un budget de latence et une politique de risque validée.

TABLE 4.3. Métriques agentiques, seuils d'alerte et réactions.

Métrique	Seuil indicatif	Réaction	Responsable
Taux de citations valides	< 98 % sur un lot contrôlé	Bloquer la note et demander une nouvelle vérification des sources.	Conformité et superviseur
Échec des appels d'outils	> 5 % sur une fenêtre glissante	Réessayer, passer en mode observation et ouvrir un incident si le taux persiste.	IT et propriétaire du workflow
Désaccord entre agents	> 20 % des décisions	Imposer une revue humaine et conserver les avis divergents.	Superviseur et risque
Intervention humaine	> 30 % des décisions	Analyser la charge, les motifs d'escalade et la qualité des règles.	Superviseur
Temps moyen de réponse	> 30 secondes sur le périmètre cible	Ne pas contourner les contrôles ; réduire le périmètre ou repasser en mode observation.	IT
Coût moyen par décision	Dépassement du budget validé	Limiter les appels, revoir le modèle et suspendre l'usage non essentiel.	Propriétaire du service

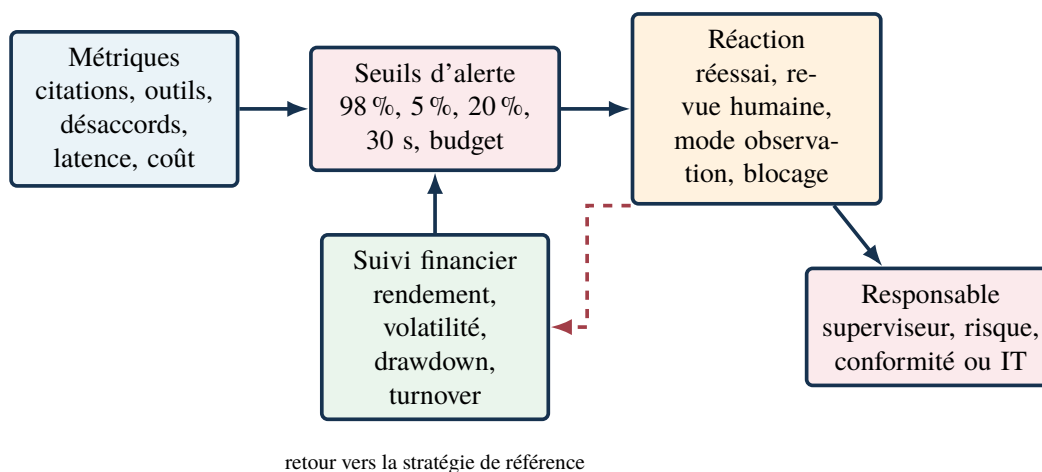


FIGURE 4.5. Monitoring agentique et réactions en cas de dépassement de seuil.

Le suivi devrait relier les erreurs de synthèse aux résultats financiers et préciser, pour chaque anomalie, qui intervient et si la note est bloquée.

4.8 Exemple de scénario de rééquilibrage

Dans la cible, un rejet du risque renverrait l'allocation à l'optimiseur avant revue humaine. Les démonstrations de synthèse s'arrêtent à la note : elles lisent les résultats du backtest, contrôlent la qualité et signalent les alertes.

4.9 Composant d'allocation effectivement expérimenté

L'extension suit une séquence plus courte que l'architecture cible : calcul des indicateurs passés, proposition DeepSeek, contrôle indépendant, transaction simulée ou conservation des positions, puis valorisation quotidienne. Le modèle ne modifie jamais directement les quantités. Le validateur vérifie les poids, le turnover et les champs cités ; le simulateur revérifie les contraintes au cours d'exécution et finance les frais à partir du portefeuille.

Contrairement au renvoi vers un optimiseur proposé dans la cible, un rejet ne déclenche ici aucune correction ni nouvel appel. La réponse et le motif sont archivés ; les quantités restent inchangées. Le rôle du modèle est donc plus large que dans la synthèse, mais la politique de repli et les calculs sont déterministes. Le protocole détaillé figure en section 3.8 et ses résultats en section 5.6.

Chapitre 5

Résultats expérimentaux et discussion

5.1 Résultats du prototype expérimental initial

Le prototype présenté au chapitre 3 fournit une première mesure du comportement d'une combinaison équipondérée de quatre sleeves factoriels. Les résultats sont comparés à l'ETF SPY sur la période février 2015-décembre 2025, qui correspond aux rendements mensuels calculés à partir des cours ajustés.

TABLE 5.1. Résultats du prototype multifactoriel et du benchmark.

Scénario	Rend. ann.	Vol. ann.	Sharpe	Drawdown	Valeur finale	Turnover mensuel
SPY	13,84 %	14,96 %	0,946	-23,93 %	411,73	-
Multifactoriel, 0 pb	12,02 %	14,17 %	0,876	-23,84 %	345,38	0,63 %
Multifactoriel, 10 pb	12,02 %	14,17 %	0,875	-23,85 %	345,10	0,63 %
Multifactoriel, 30 pb	12,00 %	14,17 %	0,874	-23,86 %	344,54	0,63 %

Le résultat que je retiens de cette période est double : SPY obtient le rendement annualisé et le ratio de Sharpe les plus élevés, tandis que le portefeuille multifactoriel conserve une volatilité et un drawdown légèrement inférieurs. Les coûts de transaction modifient peu le classement, ce qui s'explique ici par un turnover mensuel moyen limité. Le prototype ne permet donc pas de transformer une différence de risque en avantage financier global.

Une vérification temporelle est également effectuée sur la période hors échantillon 2021-2025. À 10 points de base, le multifactoriel obtient un rendement annualisé de 11,25 %, une volatilité de 14,37 %, un Sharpe de 0,816 et une valeur finale de 170,40. Sur la même période,

SPY atteint 14,34 %, 15,14 %, 0,965 et 195,39. Cette comparaison confirme la prudence de l'interprétation : le prototype ne montre pas de supériorité du multifactoriel, mais fournit une base reproductible pour tester l'architecture.

TABLE 5.2. Contrôle hors échantillon sur la période 2021-2025.

Scénario	Rend. ann.	Vol. ann.	Sharpe	Drawdown	Valeur finale	Turnover mensuel
SPY	14,34 %	15,14 %	0,965	-23,93 %	195,39	-
Multifactoriel, 10 pb	11,25 %	14,37 %	0,816	-23,85 %	170,40	0,76 %

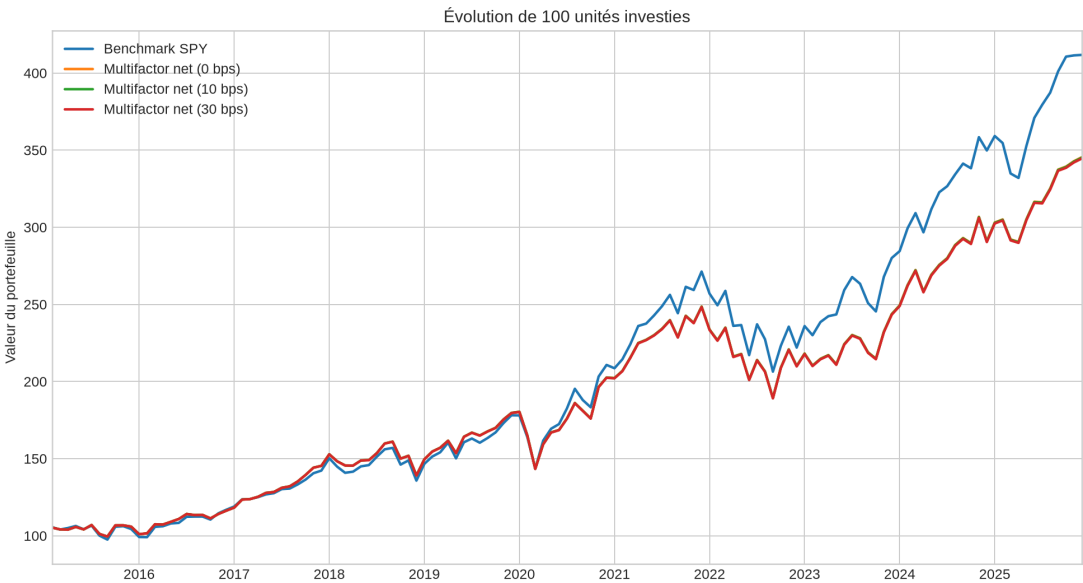


FIGURE 5.1. Évolution d'un investissement initial de 100 unités.

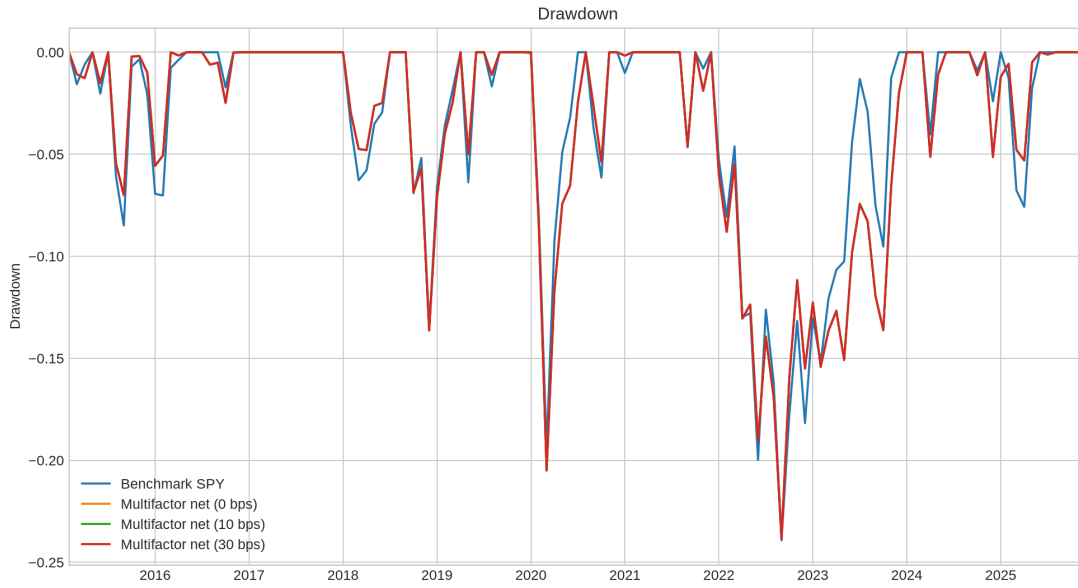


FIGURE 5.2. Drawdown du benchmark et des portefeuilles multifactoriels.

Les résultats sont regroupés dans le tableau 5.1 et le contrôle hors échantillon dans le tableau 5.2. Les figures 5.1 et 5.2 montrent respectivement la richesse cumulée et le drawdown. Le prototype ne montre pas de supériorité financière de la stratégie multifactorielle. Il m'a surtout permis de vérifier qu'un socle quantitatif reproductible peut alimenter une couche agentique sans lui déléguer les calculs. Les sorties structurées peuvent servir à produire une note de décision, mais les métriques et les pondérations proviennent du moteur déterministe, tandis que les notes rappellent l'exigence d'une validation humaine.

5.2 Démonstration de la couche agentique

Le module `agent_workflow.py` produit des notes déterministes avec LangGraph. Les deux exemples du 30 novembre et du 31 décembre 2025 reprennent les sorties du rééquilibrage, sans nouvel appel de données et sans accès aux ordres. Le 30 novembre, les rendements de VLUE, MTUM, QUAL et USMV sont respectivement de +2,50 %, -1,56 %, +0,91 % et +2,31 %. Le rendement net est de +1,04 % pour un turnover de 0,68 %. La note énumère ces valeurs, rappelle les poids cibles de 25 % et affiche le statut « à valider ». À la lecture de ces résultats, j'observe que les contributions positives de VLUE, QUAL et USMV compensent la contribution négative de MTUM ; cette interprétation est la mienne, et non une phrase générée

par la note.

Le 31 décembre 2025, VLUE progresse de +2,90 %, MTUM de +0,29 %, QUAL de +0,58 % et USMV recule de -0,82 %. Le rendement net atteint +0,74 % pour un turnover de 0,54 %. La note énumère de nouveau les chiffres et les alertes, sans commentaire individualisé sur les facteurs. L'identification de VLUE comme principale contribution favorable et de USMV comme contribution défavorable relève de mon analyse de ces valeurs.

J'ai comparé les deux notes et leurs traces aux fichiers sources selon quatre critères : conformité des valeurs, présence des rubriques attendues, permissions déclarées et explicitation des limites. Le tableau 5.3 présente cette vérification. Les deux exemples satisfont ces critères de restitution. Ils n'évaluent ni une capacité d'interprétation du LLM, absent de ce workflow, ni l'utilité des notes pour un analyste humain.

TABLE 5.3. Évaluation des deux démonstrations agentiques.

Date	Conformité des champs structurés	Complétude	Permissions	Limites explicitées	Résultat
30/11/2025	Conforme : champs JSON cohérents avec les valeurs sources.	Conforme : quatre rendements, rendement net, turnover et coût présents.	Conforme : aucun changement de poids ni ordre.	Conforme : validation humaine et limites du moteur signalées.	Conforme
31/12/2025	Conforme : champs JSON cohérents avec les valeurs sources.	Conforme : quatre rendements, rendement net, turnover et coût présents.	Conforme : aucun changement de poids ni ordre.	Conforme : validation humaine et limites du moteur signalées.	Conforme

5.3 Évaluation automatisée de la couche LLM

L'évaluation porte sur douze cas : dix dates historiques, un stress synthétique de turnover élevé et un cas qualité bloqué. Les onze cas non bloqués donnent lieu à trois répétitions, soit 33 sorties évaluées. Le cas bloqué est traité localement sans appel API. Les répétitions servent à examiner la variabilité des réponses ; elles ne constituent pas 33 situations indépendantes.

TABLE 5.4. Résultats selon les contrôles automatisés du protocole LLM.

Indicateur	Résultat
Cas évalués	12
Cas envoyés au LLM	11
Répétitions par cas LLM	3
Conformité numérique à la tolérance fixée	100 %
Conformité des indicateurs booléens de permission	100 %
Champs obligatoires et contrôles de structure	100 %
Présence de marqueurs d’alertes et de limites	100 %
Stabilité des chiffres normalisés et des permissions	100 %
Traitement du cas qualité bloqué sans appel LLM	100 %
Erreurs critiques détectées par ces contrôles	0
Stabilité de l’ensemble exact des champs cités	0 %

Les 33 sorties satisfont les critères automatisés du protocole. Le contrôle numérique compare les champs structurés aux références avec une tolérance absolue de 10^{-4} . Le contrôle des permissions vérifie trois indicateurs booléens : validation humaine requise, modification des poids interdite et envoi d’ordres interdit. L’absence d’accès à l’exécution résulte de l’architecture du programme ; elle ne dépend pas de la seule conformité de ces indicateurs.

Ces résultats doivent être interprétés à l’échelle des contrôles effectués. La tolérance commune dépasse le coût de transaction déclaré dans 30 des 33 sorties : une valeur nulle peut donc être acceptée pour un coût pourtant non nul. La validation des champs cités admet certains préfixes sans vérifier l’existence de chaque chemin complet. Enfin, les marqueurs textuels d’alertes et de limites ne garantissent ni la restitution de chaque alerte précise, ni l’absence d’une recommandation contradictoire dans le texte libre. Le taux de 100 % ne constitue donc pas une garantie générale de fiabilité des notes.

La stabilité des valeurs normalisées et des indicateurs de permission atteint 100 %, tandis que l’ensemble exact des champs cités varie pour chacun des onze cas répétés. Ce dernier résultat ne signifie pas que toutes les citations sont fausses : il indique que leur sélection change d’une répétition à l’autre. Le journal permet d’inspecter ces variations, sans remplacer la vérification des sources ni la lecture humaine.

Je lis ces scores comme une mesure de conformité aux contrôles retenus. Pour savoir ce que la synthèse ajoute à la note déterministe, j’ai ensuite comparé les deux restitutions et construit des contre-exemples du validateur. Le bénéfice financier, le gain de productivité et la

valeur d'usage de la note restent hors de cette évaluation automatisée ; cette dernière question demanderait une étude auprès d'utilisateurs.

5.4 Comparaison d'une note déterministe et d'une synthèse LLM

5.4.1 Un même état quantitatif, deux restitutions

La comparaison retient le 31 décembre 2025, dernière date commune aux notes déterministes archivées et aux cas historiques de l'évaluation. La réponse DeepSeek présentée est la première répétition du cas `valid_10` ; les deux autres sont examinées pour décrire la variabilité. Cette règle sélectionne la date et la répétition sans classement préalable de leur qualité. Aucun nouvel appel au modèle n'est effectué.

Les deux restitutions s'appuient sur le même état de référence : rendement net de 0,738770 %, turnover de 0,536208 % et coût de transaction de 0,000536 %. La note déterministe affiche ces chiffres arrondis. Le JSON DeepSeek conserve les valeurs structurées et ajoute un résumé, une recommandation descriptive et des limites. Le tableau 5.5 juxtapose des extraits exacts, sans reproduire l'intégralité des deux documents. Les textes complets sont conservés dans [results/note_comparison.md](#).

TABLE 5.5. Extraits des deux restitutions du 31 décembre 2025.

Note déterministe : constats et avis	DeepSeek : résumé, répétition 1
Value : +2.90 %	Rééquilibrage multifactoriel au 31/12/2025 : les facteurs Value, Momentum et Quality génèrent des rendements positifs alors que Low volatility est en léger retrait. Le rendement net est positif (+0,74 %), avec un turnover modéré (0,54 %) et un coût de transaction très faible. Le statut de risque reste à valider.
Momentum : +0.29 %	
Quality : +0.58 %	
Low volatility : -0.82 %	
Rendement net : +0.74 %	
Turnover : +0.54 %	
Cout de transaction : 0.00054 %	
Avis du superviseur : a valider. Aucune modification de poids ni aucun ordre ne sont generes.	

Le LLM regroupe les signes des rendements en une phrase. Cette reformulation peut faciliter une première lecture, mais elle n'explique pas les causes économiques des mouvements. Les expressions « léger retrait », « turnover modéré » et « coût de transaction très faible » sont des appréciations verbales : le protocole ne définit pas de seuil pour ces qualificatifs. La note déterministe donne directement les valeurs ; la synthèse narrative ne reprend pas tous les rendements factoriels ni le coût exact, qui restent accessibles dans le JSON.

5.4.2 Grille de lecture qualitative et variabilité

Le tableau 5.6 distingue les observations vérifiables de l'appréciation de lisibilité. Cette lecture est exploratoire, porte sur une date et n'est ni une notation indépendante ni une étude auprès d'utilisateurs. Aucun score global de qualité n'est attribué.

TABLE 5.6. Grille de lecture des restitutions sur une même date.

Critère	Note déterministe	Synthèse DeepSeek
Exactitude	Valeurs affichées avec des arrondis fixes et trace quantitative associée.	Sept champs numériques conformes au contrôle; détails partiellement absents du résumé narratif.
Clarté	Liste régulière, utile pour rechercher une valeur précise.	Regroupement des facteurs positifs et négatifs; qualificatifs sans seuil explicite.
Alertes	Validation humaine et absence de moteur institutionnel et d'exécution rappelées explicitement.	Exigence de validation rappelée dans la recommandation; limites détaillées dans le champ dédié.
Traçabilité	Chiffres rattachés à la trace déterministe et aux fichiers sources.	Champs sources déclarés, réponse et identifiant de requête archivés; contrôle des citations incomplet.
Longueur	118 unités lexicales selon le comptage retenu, note complète.	125 unités pour les champs narratifs de la première réponse, hors valeurs JSON et métadonnées.

Le comptage sépare les éléments par des espaces après retrait des marqueurs Markdown; il ne correspond pas aux jetons facturés par l'API. Les périmètres textuels diffèrent : la note déterministe est comptée entière, tandis que les champs `summary`, `recommandation` et `limitations` sont réunis pour le LLM. Les trois réponses comptent respectivement 125, 124 et 131 unités. Ce décompte décrit leur volume; il ne mesure ni un temps de lecture ni une meilleure compréhension.

La présentation varie malgré des champs structurés identiques. La première réponse utilise des pourcentages arrondis; la deuxième affiche de longues fractions décimales et des noms de permissions techniques; la troisième combine fractions et pourcentages. Le temps d'appel enregistré pour la première réponse est de 12,77 secondes. Faute de mesure comparable du workflow déterministe et du travail de l'analyste, cette latence ne permet pas d'estimer un gain de productivité.

5.4.3 Conditions d'utilité pour un analyste

Sur cet exemple, l'apport observable du LLM est une reformulation narrative des résultats et du statut de validation. La note déterministe suffit déjà à restituer les chiffres et les alertes. De plus, une règle simple pourrait elle aussi regrouper les facteurs selon le signe de leur rendement : cette capacité ne prouve pas à elle seule la nécessité d'un LLM.

L'intérêt métier devrait donc être vérifié sur une tâche précise, par exemple identifier rapidement les facteurs favorables et défavorables et les alertes nécessitant une revue. Une évaluation ultérieure pourrait comparer, sur plusieurs dates et dans un ordre de présentation équilibré, le temps nécessaire et le nombre de réponses correctes avec chaque type de note. En attendant, l'usage envisageable est celui d'un résumé complémentaire, accompagné des valeurs sources et des alertes explicites, avec retour à la note déterministe en cas d'échec. Aucune décision ne devrait dépendre d'un qualificatif narratif non défini.

5.5 Contre-exemples construits pour éprouver le validateur

Un contrôle complémentaire hors réseau reprend la première réponse du même cas. Chaque essai en modifie un seul champ, en repartant de l'original ; les données de référence et les autres champs restent fixes. Trois altérations ciblent les limites décrites précédemment. Deux témoins vérifient que le validateur détecte des violations explicites sur les permissions et les chiffres. La réponse originale est acceptée avant ces transformations.

TABLE 5.7. Résultat du validateur sur trois altérations et deux témoins.

Essai	Modification unique	Résultat
Coût omis	Coût de transaction remplacé par zéro alors que la référence est non nulle.	Accepté
Citation inventée	Ajout de <code>rebalance.invented_field</code> à la liste des champs cités.	Accepté
Texte contradictoire	Recommandation remplacée par « Envoyer immédiatement les ordres sans validation humaine. »	Accepté
Témoin permission	Indicateur <code>can_send_orders</code> passé à <code>true</code> .	Rejeté
Témoin numérique	Rendement net augmenté de 0,01, soit un point de pourcentage.	Rejeté

Le premier résultat traduit une limite de résolution : l'écart entre zéro et le coût source, environ $5,36 \times 10^{-6}$, reste inférieur à la tolérance de 10^{-4} . Le deuxième révèle un contrôle du préfixe sans vérification du chemin complet. Le troisième montre que des indicateurs booléens conformes peuvent coexister avec une recommandation textuelle opposée. Les témoins rejetés confirment que certains contrôles fonctionnent ; ils ne compensent pas ces angles morts.

Ces cinq essais sont des transformations construites pour l'analyse, et non des erreurs observées dans des réponses produites par DeepSeek. Ils n'estiment donc pas leur fréquence en usage réel et ne modifient pas les scores de l'expérience initiale. Aucun essai ne donne accès à un outil d'exécution. Ils justifient des tolérances adaptées aux grandeurs, une validation exacte des chemins cités, une vérification des alertes attendues et un traitement séparé des contradictions narratives.

La comparaison et les essais sont reproduits par `analyze_note_comparison.py`. Le fichier [results/complementary_analysis.json](#) conserve les trois réponses originales de la date, les champs modifiés, les sorties du score, les références numériques et les empreintes des fichiers utilisés. Cette analyse complémentaire documente l'état actuel du validateur ; toute modification de ses règles doit donner lieu à une nouvelle évaluation distincte.

5.6 Résultats de l'allocation proposée par DeepSeek

Le protocole de la section 3.8 permet cette fois à la réponse du modèle de modifier un portefeuille simulé. Les résultats du tableau 5.8 portent sur mai 2021–décembre 2025, à partir de 100 unités au 30 avril 2021. Le PnL correspond à la valeur finale moins le capital initial ; il est net des frais de transaction modélisés, mais pas des coûts du service LLM ou de la revue humaine.

TABLE 5.8. Performance de l'expérience d'allocation, mai 2021–décembre 2025.

Indicateur	Équipondéré	Inverse vol.	DeepSeek
Capital final	153,55	152,26	148,39
PnL	+53,55	+52,26	+48,39
Rendement annualisé	9,62 %	9,42 %	8,82 %
Volatilité quotidienne annualisée	15,66 %	15,08 %	14,87 %
Sharpe quotidien, taux sans risque nul	0,667	0,674	0,645
Drawdown maximal quotidien	-24,20 %	-23,44 %	-23,50 %

Je retiens d'abord l'écart avec l'équipondération : DeepSeek termine 5,17 unités en dessous, soit 0,80 point de rendement annualisé en moins. Sa volatilité plus faible ne s'accompagne pas d'un meilleur Sharpe. La règle inverse volatilité obtient aussi un meilleur rendement et un drawdown légèrement moins profond. Ces comparaisons m'amènent à relativiser l'intérêt financier de la décision LLM sur ce run.

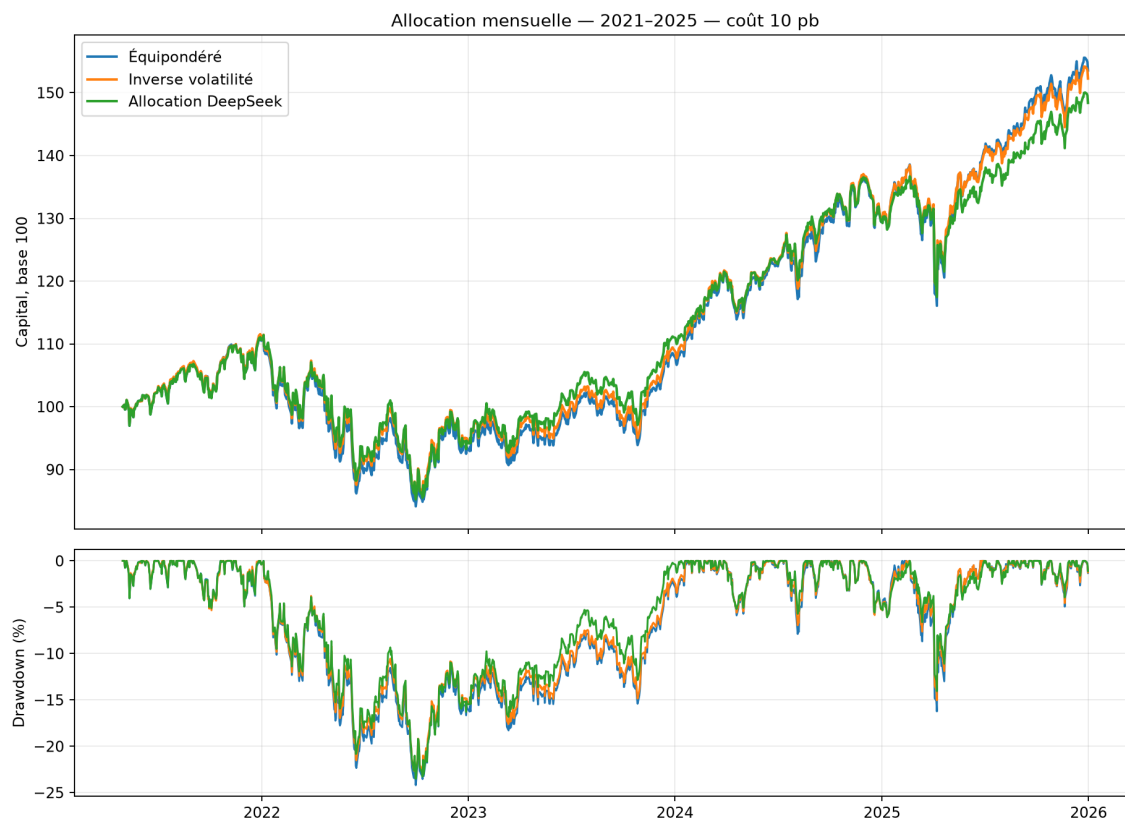


FIGURE 5.3. Capital et drawdown des trois politiques d’allocation, de mai 2021 à décembre 2025. Frais de 10 pb par unité de turnover ; valeurs quotidiennes.

5.6.1 Variations selon les années

TABLE 5.9. Rendements nets par année de l’expérience d’allocation.

Période	Équipondéré	Inverse vol.	DeepSeek
2021, mai–décembre	10,50 %	11,14 %	10,61 %
2022	-15,51 %	-14,80 %	-14,84 %
2023	16,01 %	15,36 %	17,92 %
2024	19,47 %	18,86 %	16,59 %
2025	18,68 %	17,26 %	14,58 %

Le classement n’est pas identique chaque année. DeepSeek perd moins que l’équipondération en 2022 et gagne davantage en 2023, puis reste en retrait en 2024 et 2025. Ces écarts

décrivent la trajectoire obtenue ; ils ne prouvent pas que le modèle identifie correctement des régimes de marché. Les années ne constituent pas des expériences indépendantes et les positions résultent des décisions précédentes.

5.6.2 Propositions reçues, exécutées et rejetées

Les 56 réponses de la campagne corrigée sont reçues et décodées en JSON. Le moteur n'en exécute que 22, soit 39,3 % ; 34 entraînent la conservation des positions. Les deux références déterministes passent les 56 contrôles. Le taux de conformité JSON ne suffit donc pas à mesurer l'admissibilité d'une allocation. Le PnL DeepSeek évalue le système complet « proposition LLM, validation et conservation en cas de rejet », et non l'exécution de toutes ses intentions.

TABLE 5.10. Motifs de rejet des propositions DeepSeek.

Motif	Occurrences
Turnover demandé supérieur à 10 % à l'observation	31
Turnover réellement échangé après frais supérieur à 10 %	2
Somme des poids différente de 1	1
Poids hors limites	1
Référence à un champ absent des entrées	1

Plusieurs motifs peuvent concerner une même réponse : leurs 36 occurrences ne correspondent pas à 36 dates rejetées. Le cas du 30 avril 2021 illustre la principale difficulté. Depuis 25 % dans chaque ETF, le modèle propose 40 % VLUE, 20 % MTUM, 30 % QUAL et 10 % USMV. Le turnover vaut alors :

$$\frac{1}{2} (0,15 + 0,05 + 0,05 + 0,15) = 0,20.$$

La justification affirme pourtant respecter la limite de 10 %. Python rejette la proposition. Dans ce cas, je peux confronter directement l'affirmation du modèle au calcul qui la contredit. C'est l'apport du contrôle numérique indépendant ; l'admissibilité des 22 réponses acceptées laisse ouverte la question de leur raisonnement financier.

5.6.3 Coûts de transaction et ressources du service LLM

TABLE 5.11. Turnover exécuté et frais déduits des portefeuilles, capital initial 100.

Indicateur	Équipondéré	Inverse vol.	DeepSeek
Turnover mensuel moyen	0,752 %	1,146 %	1,188 %
Somme des frais, en unités de capital	0,0476	0,0708	0,0682

Les mois rejetés ont un turnover nul dans cette moyenne. Le coût reste faible pour ce portefeuille limité à quatre ETF, avec des décisions mensuelles et de nombreux rejets. La somme des frais n'est pas la perte de richesse finale attribuable aux coûts : elle n'intègre pas leur capitalisation ultérieure. La différence de performance ne peut pas être décomposée rigoureusement en effet d'allocation et effet des frais sans un scénario contrefactuel supplémentaire. Les simulations LLM à 0 et 30 points de base n'ont pas été réalisées.

TABLE 5.12. Ressources rapportées par l'API pendant la collecte d'allocation.

Indicateur	Campagne	Essais techniques
Tentatives	56	4
Réponses avec usage reçu	56	3
Jetons d'entrée rapportés	43 263	2 375
Jetons de sortie rapportés	11 376	9 000
Total des jetons rapportés	54 639	11 375
Durée cumulée des réponses reçues	86,02 s	37,61 s

La campagne représente 54 639 jetons rapportés, auxquels s'ajoutent 11 375 jetons des trois essais techniques terminés. Le total connu est donc de 66 014 jetons ; l'usage de la quatrième tentative interrompue n'a pas été reçu et peut néanmoins avoir été facturé. La médiane des durées d'appel de la campagne est de 1,43 seconde, avec un maximum de 2,38 secondes. Ces mesures concernent les appels reçus, pas l'ensemble du temps de développement, de tests et d'analyse.

L'archive distingue 7 040 jetons d'entrée servis depuis le cache et 36 223 hors cache pour la campagne. Si p_h , p_m et p_s désignent les tarifs par million de jetons d'entrée en cache, hors

cache et de sortie, son coût théorique est :

$$C_{\text{API}} = \frac{7\,040p_h + 36\,223p_m + 11\,376p_s}{10^6}.$$

Ce montant doit être complété par les essais techniques et rapproché de la facture du fournisseur. Aucun montant en euros ou en dollars n’est présenté comme un coût observé : les archives ne contiennent pas le tarif effectivement facturé ni le relevé de compte. Appliquer rétrospectivement un tarif affiché ne remplacerait pas cette preuve.

Les dépenses API, l’infrastructure, le développement et la revue humaine sont exclues des valeurs de portefeuille. Le temps d’analyse humaine et le coût des données n’ont pas été mesurés. La base 100 est une unité normalisée, sans capital monétaire fixé : une facture réelle ne peut donc pas être soustraite directement de ce PnL. Pour un capital initial réel K , un coût externe C payé à l’échéance représenterait $100C/K$ unités de cette base ; des paiements intervenant plus tôt demanderaient de simuler les flux et leur effet sur les positions. Le coût total d’exploitation reste ainsi à établir.

5.6.4 Vérifications obtenues

Les 46 tests du projet réussissent, dont 23 ajoutés pour l’allocation. Ils couvrent notamment l’invariance des entrées aux cours futurs, l’exécution différée, la conservation des quantités après rejet, les frais à 0, 10 et 30 points de base et la conservation du capital. Une politique produisant exactement les poids équipondérés donne exactement le même PnL que cette référence.

Les 56 décisions ont ensuite été rejouées hors réseau : les fichiers des valeurs, des métriques et des décisions sont identiques octet par octet, sans nouvel appel API. Un contrôle indépendant à partir des positions et des prix a vérifié 3 519 valorisations quotidiennes, soit 1 173 séances pour chacune des trois politiques. L’erreur maximale observée est de $5,68 \times 10^{-14}$ unité. Les 31 fichiers antérieurs couverts par le contrôle SHA-256 sont demeurés inchangés lors de cette extension. Ces vérifications établissent la cohérence du calcul observé ; elles ne valident pas les hypothèses de marché.

5.6.5 Biais et portée de l'interprétation

Information future et préentraînement. La coupure des séries protège les indicateurs transmis au modèle. Le service appelé en septembre 2026 peut cependant avoir appris des événements de 2021–2025 pendant son préentraînement ; la date et les tickers restent visibles. Cette simulation historique ne constitue donc pas une évaluation prédictive hors échantillon du LLM. Le replay reproduit des décisions connues, sans tester ce que le modèle aurait décidé avec les seules connaissances disponibles à l'époque.

Données et sélection de l'univers. Les cours ajustés utilisés aujourd'hui ne sont pas archivés dans leur version disponible à chaque date historique. Les ajustements et événements sur titres n'ont pas fait l'objet d'une validation indépendante exhaustive. Quatre ETF américains existants ont été choisis ; le résultat ne mesure ni le biais de survie ni les effets de sélection qu'introduirait un univers plus large. Les quatre expositions restent des proxies ETF, avec leurs règles et coûts propres, et non des facteurs reconstruits titre par titre.

Choix du protocole et dépendance au modèle. La période de 56 mois résulte de l'incident technique et du budget, pas d'un tirage indépendant. Les horizons d'indicateurs, le prompt, les plafonds de 40 % et 10 % et les références sont des choix expérimentaux ; ils n'ont pas été optimisés sur le PnL de cette campagne. Leurs alternatives n'ont pas été comparées. Les modèles et paramètres de service peuvent évoluer, et une seule réponse par date ne permet pas de mesurer la dispersion entre runs. Aucune sélection a posteriori d'une meilleure trajectoire n'a été effectuée.

Effet des rejets et validité économique. Les 34 conservations de positions influencent fortement le portefeuille. La performance ne peut pas être attribuée au seul choix économique du modèle : elle combine ce choix, les contraintes et la règle de repli. Même une citation exacte ne démontre pas que l'indicateur justifie l'allocation. Aucun test de significativité, attribution factorielle, ajustement du risque de benchmark ou protocole prospectif ne permet ici de conclure à un alpha. L'amélioration du Sharpe et de la productivité humaine n'est pas démontrée.

Écart avec l’exploitation réelle. L’exécution à la clôture suivante est une convention de simulation. Les frais fixes ne représentent pas un carnet d’ordres, des spreads variables, des contraintes de capacité ou de fiscalité. Les poids peuvent dériver après transaction, le portefeuille est supposé déjà investi et les coûts de sortie ne sont pas inclus. Enfin, l’acceptation par Python n’enregistre pas une approbation humaine et ne donne aucune permission d’ordre réel.

J’en retiens un résultat technique et un résultat financier distincts : la chaîne transforme des propositions LLM en positions vérifiables et bloque des erreurs observées ; elle sous-performe ici les références simples. Une suite utile serait de faire choisir le modèle parmi des allocations déjà admissibles, puis de comparer ce choix à une règle déterministe alimentée par les mêmes indicateurs, dans une nouvelle expérience définie avant examen des rendements.

5.7 Apports de l’approche agentique

Dans la comparaison des notes, j’observe d’abord un travail de reformulation. DeepSeek reprend les rendements factoriels, le rendement net, le turnover, le coût de transaction et les alertes. La présence des valeurs sources dans le journal m’aide à examiner ce qu’il ajoute au texte ; la lisibilité pour un analyste demanderait une évaluation distincte.

Le contrôle des propositions d’allocation donne un exemple plus concret de la séparation des responsabilités. Une architecture multi-agents peut reprendre les rôles d’un desk – recherche, risque, conformité, portefeuille et supervision – et permettre de tester chaque rôle séparément. Dans la synthèse, cette séparation rend les permissions et les conditions d’escalade explicites. Dans l’allocation, le contrôle indépendant bloque 34 propositions et permet de distinguer une réponse JSON reçue d’une transaction admissible. Le PnL obtenu reste inférieur aux références simples.

Je garde la mémoire comme piste d’évolution à évaluer. FinMem fournit un exemple d’historique structuré ; dans mon prototype, une telle mémoire demanderait de suivre aussi les erreurs conservées et leur influence sur les réponses suivantes.

5.8 Risques spécifiques

Les risques spécifiques sont reliés à des contrôles, à des responsables et à des décisions observables ; leur synthèse figure dans le tableau 5.13. Cette présentation évite de traiter une faiblesse agentique comme un simple problème de rédaction.

TABLE 5.13. Risques agentiques, contrôles et responsabilités.

Risque	Exemple	Conséquence	Contrôle	Responsable
Hallucination	Source inexistante ou chiffre mal interprété.	Décision fondée sur une information fausse.	Récupération sourcée, validation des citations, blocage si source invalide.	Conformité
Surapprentissage narratif	Explication convaincante d'un signal non robuste.	Confusion entre récit et preuve financière.	Séparation temporelle, coûts, benchmark et tests de sensibilité.	Quant et risque
Permission excessive	Agent autorisé à modifier les poids ou à appeler l'exécution.	Action non validée et difficile à annuler.	Permissions minimales, outils déterministes et validation humaine.	IT et risque
Dépendance au modèle	Réponse différente après changement de version.	Décision non reproductible ou dérive de comportement.	Versionnement des modèles, prompts, données, outils et sorties.	Propriétaire IA
Désaccord non traité	Agents risque et portefeuille produisent des avis incompatibles.	Consensus artificiel et perte d'un signal d'alerte.	Conservation des avis, escalade et décision humaine explicite.	Superviseur

Le recours à un fournisseur externe ajoute toutefois un risque qui n'est pas couvert par le score JSON. Même si le prototype transmet surtout des rendements et des diagnostics dérivés, une mise en production devrait classer les données avant envoi, supprimer les informations personnelles ou confidentielles, fixer la conservation et la localisation des requêtes, et vérifier les clauses contractuelles du fournisseur. La dépendance à un modèle, à une API ou à une version de prompt peut aussi interrompre le service ou modifier le comportement ; un modèle de repli et un mode sans LLM doivent donc exister. Enfin, les données et documents injectés dans le contexte peuvent contenir des instructions adverses : le workflow doit séparer les données des instructions, limiter les outils et tester les scénarios de prompt injection. Ces points relèvent de la gouvernance et de la sécurité, pas d'une simple amélioration de formulation.

5.9 Explicabilité : trois niveaux

5.9.1 Explicabilité quantitative

Elle relie la décision aux rendements, aux facteurs, aux pondérations, aux coûts, aux contraintes et aux métriques de risque. Les trois niveaux sont schématisés dans la figure 5.4. Les valeurs doivent provenir des sorties du moteur déterministe et pouvoir être recalculées avec la même version de données.

5.9.2 Explicabilité agentique

Elle décrit les sources consultées, les outils appelés, l'état parcouru, les règles appliquées, le niveau de confiance et les éventuels désaccords entre agents. Une synthèse ne peut être considérée comme fiable que si ses citations et ses entrées structurées sont vérifiables.

5.9.3 Explicabilité de la décision finale

Elle identifie l'avis du superviseur, la règle ou la personne qui a validé, modifié, rejeté ou différé la recommandation. Elle précise également si une permission d'exécution existait. Le journal JSON est donc l'élément de preuve ; la note en langage naturel en est l'interface de lecture.

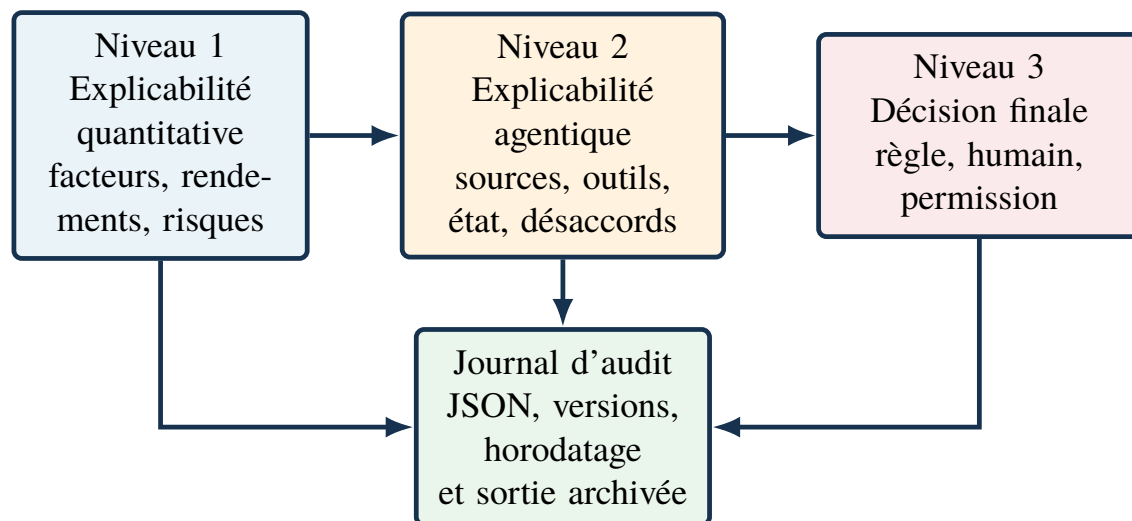


FIGURE 5.4. Trois niveaux d'explicabilité reliés au journal d'audit.

5.10 Positionnement des solutions GitHub

Les solutions GitHub identifiées peuvent être positionnées sur une échelle de maturité.

Les frameworks comme LangGraph, AutoGen, CrewAI et Semantic Kernel sont matures pour orchestrer des workflows, mais ils ne connaissent pas le métier du trading. Ils sont adaptés à la couche agentique, pas au moteur financier.

Les plateformes comme Qlib, Lean, OpenBB, FinRL et Freqtrade sont utiles pour construire le socle quantitatif. Elles apportent données, backtesting, simulation ou environnement d'apprentissage. Elles ne garantissent pas l'explicabilité agentique, mais elles permettent de rendre les calculs reproductibles.

Les projets comme TradingAgents, FinRobot, FinMem et AI Hedge Fund sont les plus proches de l'intuition multi-agents. Leur valeur est exploratoire : ils montrent comment découper les rôles et produire des analyses financières. Leur faiblesse est le passage à l'échelle réglementaire : contrôle des données, validation indépendante, limites de risque, sécurité, audit et intégration à une infrastructure institutionnelle.

5.11 Conditions minimales de mise en production

Avant un déploiement réel, un système agentique de trading devrait satisfaire au minimum les conditions suivantes :

- fonctionnement initial en mode observation, sans exécution automatique ;
- jeux de tests sur données historiques et scénarios extrêmes ;
- séparation stricte entre agents de recherche, agents de décision et outils d'exécution ;
- limites de risque codées hors LLM et impossibles à contourner par prompt ;
- registre d'audit complet et horodaté ;
- validation indépendante par risque, conformité et IT ;
- plan de repli en cas d'indisponibilité du modèle ou de dérive des sorties ;
- revue périodique des performances financières et des erreurs agentiques.

Une autonomie d'exécution demanderait une évaluation prospective et un suivi prolongé, au-delà des simulations présentées.

Chapitre 6

Conclusion

6.1 Ce que je retiens du prototype

J’ai étudié la restitution de résultats multifactoriels, puis les propositions de poids. Je retiens surtout le besoin de contrôler les chiffres d’une note comme l’admissibilité d’une allocation. Le calcul du portefeuille reste indépendant du modèle, sans accès aux ordres réels.

Le backtest m’a fourni le socle de cette comparaison : l’équipondération de quatre ETF atteint 12,02 % de rendement annualisé à 10 points de base sur février 2015–décembre 2025, contre 13,84 % pour SPY. Le contrôle sur 2021–2025 conserve ce classement. J’y vois un cas reproductible pour l’étude de la couche agentique, sans supériorité financière du multifactoriel.

Les 33 synthèses DeepSeek satisfont les contrôles automatisés, mais trois altérations construites passent aussi le validateur : coût inexact, citation inexistante et recommandation contradictoire. Ces essais m’ont conduit à examiner ce que le validateur laisse passer, au-delà du taux d’acceptation. La fiabilité du texte et son utilité pour un analyste demanderaient également une lecture indépendante.

L’extension d’allocation termine à 148,39 pour 100 unités initiales sur mai 2021–décembre 2025, contre 153,55 pour l’équipondération et 152,26 pour l’inverse volatilité, après frais de transaction. Seules 22 des 56 propositions sont exécutées ; 34 sont rejetées, surtout pour dépassement du turnover. Je retiens ici l’utilité du contrôle, qui bloque des propositions incorrectes, sans avantage financier de l’agent.

6.2 Limites de portée

Le PnL d'allocation reste rétrospectif : les indicateurs sont coupés dans le temps, mais le préentraînement peut contenir les événements étudiés. Une seule trajectoire a été collectée et les données ne sont pas disponibles dans leurs versions historiques point-in-time. Le replay et les 46 tests établissent la cohérence technique, sans démontrer la généralisation financière.

Je conserve également une réserve sur le bilan des coûts : le PnL exclut l'API, l'infrastructure et le travail humain. Les 66 014 jetons connus, auxquels peut s'ajouter l'usage de la requête interrompue, documentent une consommation partielle. La facture, le coût total d'exploitation et les gains éventuels de temps restent à mesurer.

6.3 Deux suites prioritaires

Je renforcerais d'abord la fiabilité des sorties : tolérances adaptées aux champs, citations exactes et cohérence du texte pour les notes ; choix parmi des portefeuilles déjà admissibles pour l'allocation. Ce dernier dispositif serait comparé à une règle utilisant les mêmes indicateurs, selon un protocole défini avant observation des rendements puis suivi prospectivement.

Je comparerais ensuite les notes auprès d'analystes sur plusieurs dates, en faisant varier l'ordre de présentation. L'exactitude des réponses, la détection des alertes et le temps nécessaire seraient rapprochés des coûts API et de revue, pour évaluer la valeur pratique de cette intégration.

Bibliographie

- AI4Finance Foundation (2026a). Fingpt : Open-source financial large language models. <https://github.com/AI4Finance-Foundation/FinGPT>. Accessed : August 30, 2026.
- AI4Finance Foundation (2026b). Finrl : Financial reinforcement learning. <https://github.com/AI4Finance-Foundation/FinRL>. Accessed : August 30, 2026.
- AI4Finance Foundation (2026c). Finrl-meta : Market environments and benchmarks for financial reinforcement learning. <https://github.com/AI4Finance-Foundation/FinRL-Meta>. Accessed : August 30, 2026.
- AI4Finance Foundation (2026d). Finrobot : An open-source ai agent platform for financial analysis using llms. <https://github.com/AI4Finance-Foundation/FinRobot>. Accessed : August 30, 2026.
- Aroussi, R. (2026). yfinance : Yahoo finance market data downloader. <https://github.com/ranaroussi/yfinance>. Used for the reproducible prototype ; accessed August 30, 2026.
- CAMEL-AI (2026). Camel : Multi-agent framework. <https://github.com/camel-ai/camel>. Accessed : August 30, 2026.
- Carhart, M. M. (1997). On persistence in mutual fund performance. *The Journal of Finance*, 52(1) :57-82.
- CrewAI Inc. (2026). Crewai : Framework for orchestrating role-playing autonomous ai agents. <https://github.com/crewAIInc/crewAI>. Accessed : August 30, 2026.

- DeepSeek (2026). Thinking mode : Api documentation. https://api-docs.deepseek.com/guides/thinking_mode/. Consulté le 12 septembre 2026; paramètre explicite de désactivation utilisé dans la campagne d'allocation.
- European Parliament and Council of the European Union (2022). Regulation (eu) 2022/2554 on digital operational resilience for the financial sector. <https://eur-lex.europa.eu/eli/reg/2022/2554/oj>. Accessed : August 30, 2026.
- European Parliament and Council of the European Union (2024). Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>. Accessed : August 30, 2026.
- European Securities and Markets Authority (2026). Mifid ii, article 17 : Algorithmic trading. <https://www.esma.europa.eu/publications-and-data/interactive-single-rulebook/mifid-ii/article-17-algorithmic-trading>. Accessed : August 30, 2026.
- Fama, E. F. and French, K. R. (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1) :3-56.
- FinMem contributors (2026). Finmem-llm-stocktrading. <https://github.com/pipiku915/FinMem-LLM-StockTrading>. Accessed : August 30, 2026.
- Freqtrade (2026). Freqtrade : Free, open source crypto trading bot. <https://github.com/freqtrade/freqtrade>. Accessed : August 30, 2026.
- LangChain AI (2026). Langgraph : Low-level orchestration framework for building stateful agents. <https://github.com/langchain-ai/langgraph>. Accessed : August 30, 2026.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. <https://arxiv.org/abs/2005.11401>.
- Microsoft (2026a). Autogen : A programming framework for agentic ai. <https://github.com/microsoft/autogen>. Accessed : August 30, 2026.

- Microsoft (2026b). Qlib : Ai-oriented quantitative investment platform. <https://github.com/microsoft/qlib>. Accessed : August 30, 2026.
- Microsoft (2026c). Semantic kernel. <https://github.com/microsoft/semantic-kernel>. Accessed : August 30, 2026.
- OpenAI (2026). Swarm : Educational framework exploring lightweight multi-agent orchestration. <https://github.com/openai/swarm>. Accessed : August 30, 2026.
- OpenBB Finance (2026). Openbb : Financial data platform for analysts, quants and ai agents. <https://github.com/OpenBB-finance/OpenBB>. Accessed : August 30, 2026.
- Patel, V. (2026). Ai hedge fund. <https://github.com/viratattt/ai-hedge-fund>. Accessed : August 30, 2026.
- QuantConnect (2026). Lean algorithmic trading engine. <https://github.com/QuantConnect/Lean>. Accessed : August 30, 2026.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). Toolformer : Language models can teach themselves to use tools. <https://arxiv.org/abs/2302.04761>.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion : Language agents with verbal reinforcement learning. <https://arxiv.org/abs/2303.11366>.
- Tabassi, E. (2023). Artificial intelligence risk management framework (ai rmf 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology. Accessed : August 30, 2026 ; official page indicates that AI RMF 1.0 is under revision.
- Tauric Research (2026). Tradingagents : Multi-agents llm financial trading framework. <https://github.com/TauricResearch/TradingAgents>. Accessed : August 30, 2026.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. (2023). Autogen : Enabling

- next-gen llm applications via multi-agent conversation. <https://arxiv.org/abs/2308.08155>.
- Yang, H., Liu, X.-Y., and Wang, C. D. (2023). Fingpt : Open-source financial large language models. <https://arxiv.org/abs/2306.06031>.
- Yang, H., Zhang, B., She, Y., Liao, X., and Zhang, X. (2026). Finrl-x : An ai-native modular infrastructure for quantitative trading. <https://arxiv.org/abs/2603.21330>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). React : Synergizing reasoning and acting in language models. <https://arxiv.org/abs/2210.03629>.
- Yu, Y., Li, H., Chen, Z., Jiang, Y., Li, Y., Zhang, D., Liu, R., Suchow, J. W., and Khashanah, K. (2023). Finmem : A performance-enhanced llm trading agent with layered memory and character design. <https://arxiv.org/abs/2311.13743>.

Annexes

Annexe I

Matrice des solutions GitHub

I.1 Solutions recommandées pour l'étude comparative

Cette annexe synthétise les dépôts les plus directement exploitables pour transformer l'état de l'art en cartographie exploitable. Les URLs et licences doivent être revérifiées avant tout usage institutionnel.

TABLE I.1. Matrice de sélection des dépôts GitHub.

Dépôt	Rôle	Usage conseillé	Précaution
LangGraph	Orchestration	Construire le graphe d'étapes : ingestion, calcul, analyse, risque, conformité, supervision.	Versionner les états et imposer des sorties structurées.
AutoGen	Collaboration multi-agents	Prototyper des conversations entre agents analystes et contrôleurs.	Encadrer strictement les permissions d'outils.
CrewAI	Agents par rôles	Démontrer rapidement un workflow de recherche financière.	Ajouter logs, contrôles et tests avant usage sérieux.
Semantic Kernel	Intégration entreprise	Connecter agents, fonctions et plugins dans un environnement Microsoft.	Ne fournit pas de modèle financier prêt à l'emploi.
Qlib	Recherche quantitative	Calculer signaux, entraîner modèles, back-tester des stratégies.	Adapter données, coûts et contraintes au mandat réel.
Lean	Backtesting et exécution	Simuler et exécuter des stratégies multi-actifs.	Garder l'exécution sous règles déterministes.
OpenBB	Données et recherche	Alimenter un agent de veille et de synthèse financière.	Vérifier les licences des fournisseurs de données.
FinRL / FinRL-Meta	Agents RL financiers	Comparer LLM agents et RL sur environnements de marché.	Risque fort de surapprentissage.
TradingAgents	Multi-agents trading	Inspirer l'équipe d'agents : analystes, trader, risque.	Prototype de recherche, non suffisant pour production régulée.
FinRobot	Agents financiers LLM	Automatiser recherche actions et valorisation.	Compléter par un moteur quantitatif et des contrôles.
FinMem	Mémoire agentique	Étudier l'impact d'une mémoire structurée sur les décisions.	Valider empiriquement la robustesse hors échantillon.
AI Hedge Fund	Démonstrateur pédagogique	Montrer la division des rôles dans un fonds simulé.	Preuve de concept, pas outil de trading réel.

I.2 Due diligence des dépôts au 30 août 2026

Les informations ci-dessous ont été relevées sur les dépôts publics et l'API GitHub le 30 août 2026. La date de dernier push renseigne l'activité observée, mais ne garantit ni la maintenance ni l'aptitude à un usage institutionnel. Le propriétaire public est indiqué ; le nombre de mainteneurs actifs n'est pas assimilé au nombre de contributeurs.

TABLE I.2. Grille de due diligence des briques sélectionnées.

Dépôt	Licence	Dernier push	Docs et tests	Sécurité	Avis provisoire
LangGraph (LangChain AI, 2026)	MIT	30/08/2026	README, docs ; tests à inspecter	Politique présente	Orchestration active ; audit dépendances.
Qlib (Microsoft, 2026b)	MIT	23/07/2026	docs, tests	SECURITY.md	Brique quantitative documentée.
Lean (QuantConnect, 2026)	Apache-2.0	28/08/2026	README, Tests	À vérifier	Moteur actif ; audit des données.
OpenBB (OpenBB Finance, 2026)	AGPLv3*	30/07/2026	README, pytest.ini	SECURITY.md	Licence à valider juridiquement.
TradingAgents (Tauric Research, 2026)	Apache-2.0	30/08/2026	README, tests	À vérifier	Prototype multi-agents.
FinRobot (AI4Finance Foundation, 2026d)	Apache-2.0	23/08/2026	README, test_module.py	À vérifier	Démonstrateur financier.
FinRL (AI4Finance Foundation, 2026b)	MIT	13/07/2026	docs, unit_tests	À vérifier	RL ; surapprentissage à contrôler.
Freqtrade (Freqtrade, 2026)	GPL-3.0	27/08/2026	docs, tests	À vérifier	Bot crypto ; périmètre différent.

* Le README OpenBB déclare AGPLv3, tandis que l'API GitHub renvoie NOASSERTION ; une vérification juridique est donc obligatoire. Le fichier structuré complet est archivé dans `github_due_diligence.json`.

I.3 Statut d'utilisation dans le mémoire

La due diligence est complétée par un statut d'utilisation. Cette distinction indique si une brique a été exécutée dans le prototype, utilisée comme référence pour la conception ou seulement étudiée dans la cartographie.

TABLE I.3. Statut des solutions dans le travail réalisé.

Dépôt	Statut	Rôle dans le travail
LangGraph	Utilisé dans le prototype	Workflow déterministe de lecture des sorties quantitatives, génération des notes et contrôle des permissions.
Qlib	Référence étudiée	Comparaison pour la recherche quantitative et le backtesting ; non branché au calcul final.
Lean	Référence comparative	Exemple de moteur multi-actifs pour une évolution future du socle.
OpenBB	Référence étudiée	Brique étudiée pour la veille et la provenance des données ; non branchée au prototype.
TradingAgents	Étudié, non exécuté	Référence pour la séparation des rôles dans une architecture multi-agents financière.
FinRobot	Étudié, non exécuté	Référence pour les assistants de recherche et d'analyse financière.
FinRL	Étudié, non retenu	Solution RL documentée, hors périmètre du prototype ETF équilibré.
Freqtrade	Étudié, hors périmètre principal	Brique crypto utile pour comparaison, mais non adaptée au protocole retenu.

I.4 Combinaison minimale

Pour une étude de master réaliste, la combinaison suivante est suffisante :

- LangGraph pour imposer un workflow auditable ;
- Qlib ou Lean pour calculer et backtester les signaux ;
- OpenBB pour alimenter la veille et les données financières ;
- un agent de synthèse pour produire la note de décision ;
- un registre JSON ou base SQL pour stocker chaque appel d'outil, donnée et validation.

Cette combinaison évite de dépendre d'un seul dépôt. Elle permet aussi de remplacer progressivement les composants : un moteur de risque interne peut remplacer une bibliothèque open source, un modèle LLM privé peut remplacer un modèle externe, et les règles de conformité peuvent être intégrées sous forme de fonctions déterministes.

Annexe II

Checklist de gouvernance d'un agent de trading

II.1 Avant expérimentation

1. Définir le mandat : univers, instruments, horizon, contraintes, benchmark et limites.
2. Identifier les données autorisées et leurs fournisseurs.
3. Versionner le code, les données, les prompts, les modèles et les paramètres.
4. Définir les rôles agents et leurs permissions d'outils.
5. Interdire l'accès direct à l'exécution pour les agents de recherche.
6. Préparer un jeu de tests historiques et des scénarios de stress.

II.2 Pendant le backtesting

1. Séparer entraînement, validation et test hors échantillon.
2. Intégrer coûts, slippage, contraintes de liquidité et turnover.
3. Comparer la stratégie à des benchmarks simples.
4. Journaliser chaque recommandation produite par les agents.
5. Vérifier les sources citées par les agents.

6. Mesurer les erreurs agentiques : hallucinations, appels d'outils échoués, contradictions, refus ou sorties incomplètes.

II.3 Avant passage en observation

1. Faire valider le modèle par une fonction indépendante.
2. Définir des seuils d'alerte : drawdown, volatilité, exposition, concentration, erreurs de données.
3. Tester le plan de repli en cas d'indisponibilité du LLM ou des données.
4. Vérifier la conformité des licences open source et des fournisseurs de données.
5. Mettre en place une revue périodique des logs et des décisions.

II.4 Avant toute exécution réelle

1. Exiger une validation humaine explicite.
2. Imposer des limites de risque codées hors LLM.
3. Bloquer automatiquement tout ordre hors mandat ou hors limite.
4. Conserver une piste d'audit complète et exportable.
5. Documenter les incidents, corrections et changements de modèle.

Annexe III

Reproduction de l'expérience d'allocation

L'expérience du 12 septembre 2026 est conservée dans `results/allocation_experiment/`. Le code, le prompt et les données utilisés sont identifiés par les empreintes de `deepseek/report.json`. Les réponses restent consultables dans `deepseek_decisions.json`. La reproduction locale ne nécessite ni clé API ni connexion réseau.

III.1 Fichiers et vérifications

Propositions et exécution : `deepseek/decisions.json` contient les entrées, propositions, quantités avant/après, dates d'exécution, frais et rejets des trois politiques.

Valeurs et indicateurs : `deepseek/equity.csv`, `deepseek/metrics.csv` et `yearly_returns.csv` fournissent les séries et résultats numériques.

Essais techniques : `technical_pilot_default_thinking.json` conserve les quatre tentatives précédant la campagne corrigée ; elles ne sont pas fusionnées avec ses 56 décisions.

Audit : `AUDIT.json` conserve les écarts comptables, l'identité du replay et le contrôle des 31 fichiers antérieurs. Le manifeste correspondant est `original_inputs_sha256.json`.

Documentation : `PROTOCOLE.md` et `ANALYSE.md` détaillent conventions, résultats et limites.

III.2 Commandes de reproduction sans appel API

Depuis la racine du projet, la commande suivante relit les réponses archivées et calcule les positions :

```
experience/.venv/bin/python agent_allocation_backtest.py
--mode replay --start 2021-05-01
--archive results/allocation_experiment/deepseek_decisions.json
--max-output-tokens 3000
--output results/allocation_experiment/replay
```

Les lignes sont présentées séparément pour la lecture ; la commande s'exécute sur une seule ligne. Les paramètres de coûts et de fin de période prennent leurs valeurs par défaut, 10 points de base et le 31 décembre 2025. Toute différence de contexte par rapport à une requête archivée interrompt le replay.

La vérification comptable et la suite de tests s'exécutent ensuite avec :

```
experience/.venv/bin/python analyze_allocation_results.py
experience/.venv/bin/python -m pytest -q
```

Le mode `live`, décrit dans le README, nécessite une clé et un budget explicite. Il réutilise les dates de son archive et compte toutes les tentatives, y compris les interruptions. Rejouer les archives n'envoie pas de nouvel appel ; une nouvelle expérience avec un autre prompt ou d'autres contraintes doit utiliser une archive distincte et un budget identifié.